# PACMAN

## A complete learning guide, implementation roadmap and two-person collaboration plan

Prepared for **Person 1 — Developer A** and **Person 2 — Developer B**.

| Source of truth | Guide scope |
| --- | --- |
| Pacman — Ghosts! More ghosts! Subject version 1.5 21 PDF pages (printed pages 1–20 plus cover) | Python 3.10+ · object-oriented design External maze package · graphical game Testing · packaging · defense |
| Inspected 27 October 2026 | 16 owned tasks · 108 estimated hours each Mandatory requirements separated from choices |

**Read this as a plan to implement and understand.** No game source exists in the inspected directory. Code fragments illustrate contracts and algorithms; they are not a complete game. This document does not claim that the game, release, or evaluation has been completed.

Source filename: **en.subject.pdf**, used because the requested subject.pdf is absent. The supplied mazegenerator 2.1.0 wheel was inspected without alteration. Existing inputs are preserved.

# Contents

**Contents** 2

**Chapter 1 — Understanding the Project** 5

1.1 What we are building . . . 5

1.2 The complete journey . . . 5

1.3 What is already available . . . 6

1.4 Mandatory requirement register . . . 6

1.5 Decisions and unresolved ambiguities . . . 8

1.6 Beginner preparation and proposed architecture . . . 9

**Chapter 2 — Theoretical Foundations** 10

2.1 Object-oriented design and typed boundaries . . . 10

2.2 Configuration as a parsing and validation pipeline . . . 10

2.3 Grids, bit masks and undirected graphs . . . 11

2.4 External generation, trees, cycles and adapters . . . 11

2.5 Breadth-first search and distance fields . . . 12

2.6 Randomness and reproducible debugging . . . 13

2.7 Finite-state machines and transition ownership . . . 13

2.8 Game loops, fixed steps and pause-safe clocks . . . 14

2.9 Movement, contacts and deterministic resolution . . . 15

2.10 Collections, scoring and idempotent events . . . 15

2.11 Persistent storage and atomic replacement . . . 16

2.12 Graphics, input and resource ownership . . . 16

2.13 Packaging, dependency isolation and reproducibility . . . 17

2.14 Testing, invariants and failure boundaries . . . 17

**Chapter 3 — Work Distribution** 19

3.1 Dependencies and parallel work . . . 20
3.2 Shared work and balance . . . 20
3.3 Contracts to freeze before parallel coding . . . 21

**Chapter 4 — Detailed Implementation Tasks 22**

Task A01 — Contracts, environment and quality gate . . . 22
Task A02 — CLI and robust configuration . . . 23
Task A03 — External maze adapter and validation . . . 25
Task A04 — Level assembly and reachable placement . . . 26
Task A05 — Player movement and simulation stepping . . . 27
Task A06 — Pickups, contacts, lives and score . . . 28
Task A07 — Progression, clocks and session lifecycle . . . 30
Task A08 — Management, README and defense evidence . . . 31
Task B01 — Graphics feasibility and lifecycle . . . 32
Task B02 — World renderer and HUD . . . 34
Task B03 — Input routing, menus and pause views . . . 35
Task B04 — Ghost navigation and timed states . . . 36
Task B05 — Persistent top-ten repository . . . 37
Task B06 — End screens, name entry and review cheats . . . 39
Task B07 — Acceptance harness and failure audit . . . 40
Task B08 — Packaging and platform delivery . . . 41

**Chapter 5 — Project Roadmap 43**

5.1 Phases, parallel work and exit gates . . . 43
5.2 Visual timeline . . . 44
5.3 Daily execution and risk control . . . 45

**Chapter 6 — Collaboration and Integration 46**

6.1 Branch strategy and module boundaries . . . 46

6.2 Define contracts before depending on behavior . . . 46
6.3 A concrete two-person integration example . . . 47
6.4 Review checklist and conflict handling . . . 47
6.5 Understanding the complete system . . . 47

**Chapter 7 — Testing and Validation** **48**

7.1 Test layers and evidence format . . . 48
7.2 Requirements-to-tests traceability . . . 49
7.3 Concrete configuration and graph cases . . . 50
7.4 Interaction and timing truth table . . . 51
7.5 Debugging method for difficult failures . . . 51
7.6 Evaluation rehearsal . . . 52
7.7 Final project validation checklist . . . 52

**Chapter 8 — Understanding and Defending Our Work** **53**

8.1 Core design questions and answers . . . 53
8.2 Complexity and trade-offs at a glance . . . 56
8.3 Small-change rehearsal exercises . . . 56
8.4 Sources and further learning . . . 57
8.5 Guide audit and handoff . . . 58

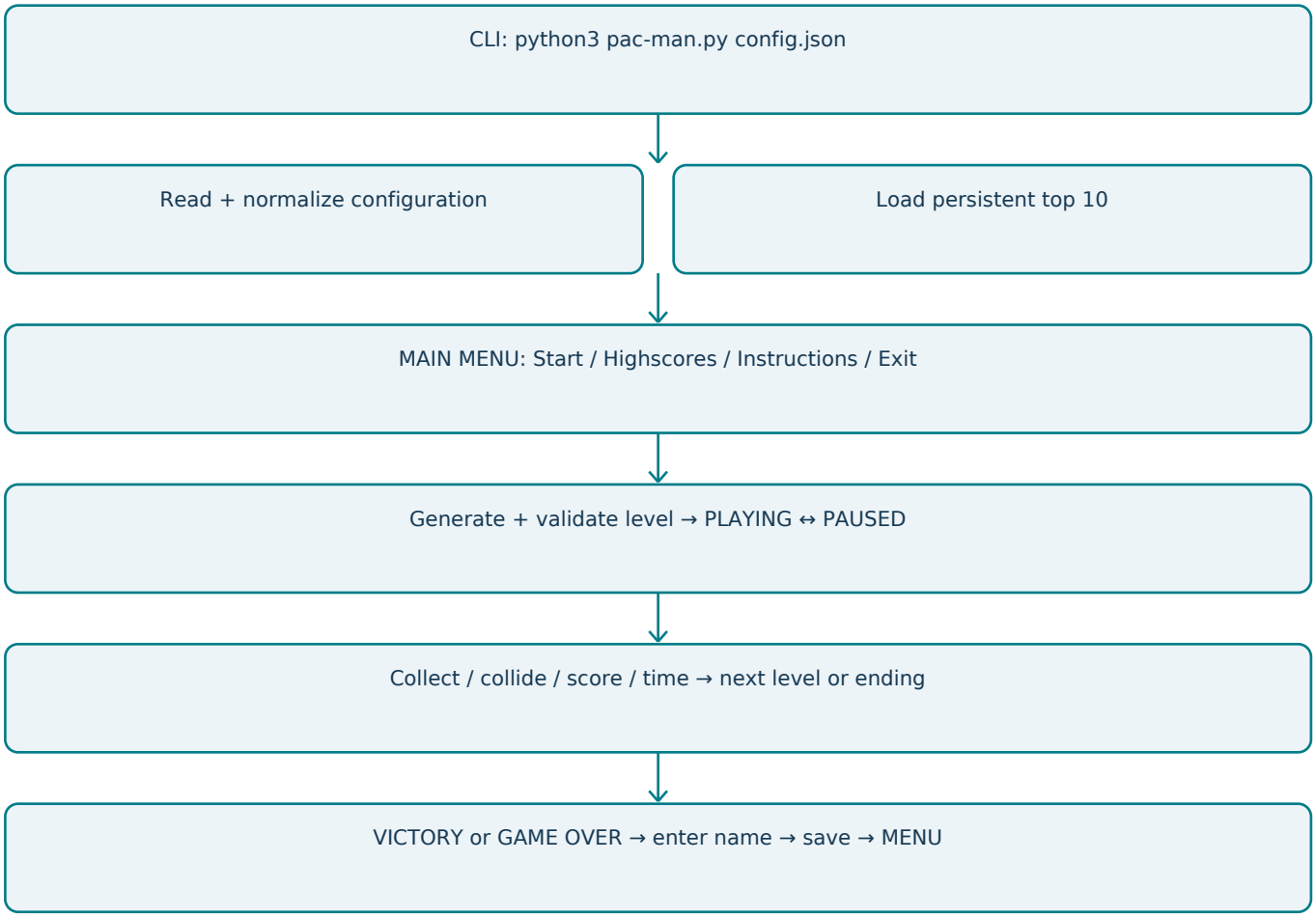
# Chapter 1 — Understanding the Project

## 1.1 What we are building

Build a complete, playable Pac-Man game in Python. The player travels through a maze, collects dots, avoids four ghosts, and uses larger power pellets to turn the chase around. The game must contain at least ten levels, preserve score and remaining lives between them, and end with a name-entry screen on victory or defeat. It must also be installable and launchable from a public gaming platform as a free unlisted/private build.

The engineering problem is larger than moving a character. Untrusted configuration, somebody else's maze package, graphics, timed simulation, disk storage, and release packaging must behave as one reliable application. The educational purpose is to practice reusable object-oriented software, controlled integration, teamwork, and explaining design decisions. Authentic arcade ghost personalities, exact artwork, sound, and a 256-level recreation are not required.

## 1.2 The complete journey




A normal session starts by reading configuration and scores, before presenting the menu. Start creates a new session with score zero and the initial lives. Level 1 uses a fixed positive seed; later levels use fresh random seeds. Every gameplay update reads intent, advances the simulation, resolves interactions, and produces a picture. Menus and paused states still accept input but do not advance game time. After a terminal result, the player enters a valid name, storage is attempted once, and the menu shows the updated leaderboard or a clear save warning.

## 1.3 What is already available

| Artifact | Finding / implication |
|---|---|
| en.subject.pdf | Entire 21-page subject read, including AI instructions, packaging, management, README, and peer review. References below use the printed page numbers. |
| mazegenerator-2.1.0-py3-none-any.whl | Only implementation artifact found. Contains MazeGenerator and metadata; no project source, config, README, Makefile, assets, tests, or Git directory found. |
| Observed API | MazeGenerator(size=(15, 15), perfect=False, entry_cell=(0, 0), exit_cell=(-1, -1), seed=42). Construction generates immediately. Read generator.maze. |
| Verified example | 15×15 / seed 42 / perfect=False: 207 reachable cells, 18 fully walled cells, center (7,7) has wall mask 10; all corners reachable; no mismatched internal walls. A second construction matches. |

Do not use the wheel's quick-start call with width= and height=: it contradicts the actual constructor. Coordinates in the implementation are (x, y); rows are indexed maze[y][x]. The metadata also describes entry/exit as (row, col), so the executable implementation takes precedence. These observations establish a working adapter, not a guarantee for every possible size or a replacement package.

## 1.4 Mandatory requirement register

**M** means mandatory. **D** means a documented design decision proposed by this guide. **O** means optional. There is no separate bonus chapter or bonus scoring rubric in the supplied subject. Suggested key names, score amounts, timer values, and cheat features are examples, not individually mandated constants. The following IDs connect ownership and acceptance tests throughout the guide.

| ID | Mandatory requirement | Subject p. | Owner task |
|---|---|---|---|
| R01 | Python 3.10+, OOP, modular/reusable playable game; flake8; typed parameters/returns/variables as applicable; mypy; PEP 257 docstrings. | 5, 7 | A01/B01 |
| R02 | Graceful exceptions and resource cleanup; no user-facing traceback or review crash; .gitignore; create edge-case test programs (not submitted or graded). | 5–6 | A01/B07 |
| R03 | Makefile: install, run, debug, clean, lint with the exact prescribed commands/flags. lint-strict optional. | 5–6 | A01 |
| R04 | Exactly one config argument: python3 pac-man.py config.json; JSON filename; clear handling of absent/bad files, values and keys. | 8–9 | A02 |
| R05 | Ignore lines starting #; document schema/defaults; missing/invalid values normalize to safe defaults with messages; unknown keys ignored. | 8–9 | A02 |
| R06 | Use assigned external generator as-is, never implement/modify it; adapt loader; PERFECT=False; clean failure; survive reinstall. | 9 | A03 |
| R07 | Fixed first-level seed, random later mazes; corridor dots; four corner power pellets and four corner ghosts; player starts centrally. | 11 | A03/A04 |
| R08 | Four-direction corridor-only movement using arrows or WASD; initially 3 lives; life lost on ghost contact; central respawn; zero lives loses. | 12 | A05/A06 |
| R09 | Autonomous ghosts chase normally, flee while edible, and return to their own corners after a delay when eaten. | 12 | B04 |

| ID | Mandatory requirement | Subject p. | Owner task |
|---|---|---|---|
| R10 | Dots/power pellets/edible ghosts add X/Y/Z points; power effect temporary; score never decreases. | 12, 14 | A06/B04 |
| R11 | At least 10 levels; per-level time limit; chosen timeout behavior; all dots clears level; all levels wins; retain score/lives; pause/resume. | 12, 14 | A07 |
| R12 | Persistent top 10: valid names ≤10 alphanumeric/space characters; non-negative integer scores; load at start, save at end; robust files; entry on win/lose. | 9–10 | B05/B06 |
| R13 | Menu Start, View Highscores, Instructions, Exit; HUD score/lives/level/time; Pause Resume and Return; final score/name on loss and victory. | 15 | B02/B03/B06 |
| R14 | Activatable cheat mode that genuinely helps review. Listed powers are suggestions. | 13 | B06 |
| R15 | Every used graphical-library function must have an MLX equivalent; MLX itself or similar library allowed. | 7 | B01/B02 |
| R16 | Fully functional free unlisted/private platform build; in-package controls/options/config instructions; full source + packaging script/spec at repository root; regenerate if asked. | 16 | B08 |
| R17 | Structured project management and evidence documents in dedicated repository subdirectory. Examples are suggested formats. | 17 | A08 |
| R18 | English root README with exact italicized attribution first line; Description, Instructions, Resources + AI usage, Configuration, Highscore, Maze Generation, Implementation, General Software Architecture, Project Management + link. | 18–19 | A08 |
| R19 | Submit repository work; be ready for config edits, generator reinstall, package rebuild and possible brief code change; understand and peer-review AI assistance. | 3–4, 9, 16, 20 | A08/B07/B08 |

## 1.5 Decisions and unresolved ambiguities

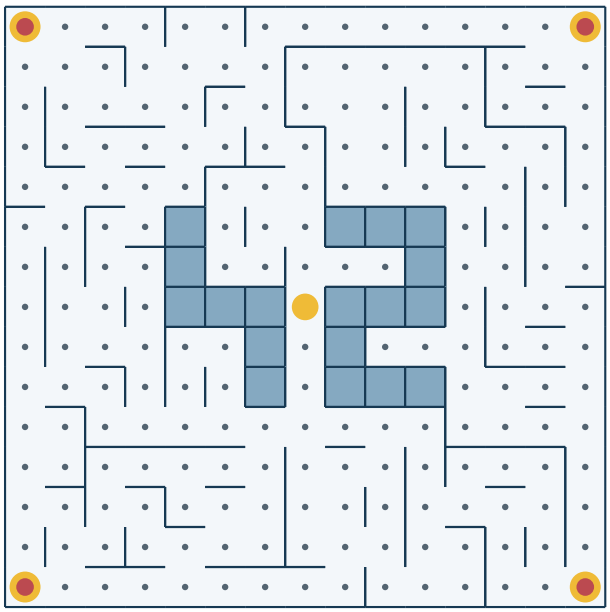
| Question | Proposed policy (D), with limit |
|---|---|
| 3 lives vs configurable lives | Default to 3. Honor a valid configured lives value; the subject both lists lives as configurable and says the player starts with 3. Confirm whether evaluators require an invariant 3. Keep the shipped default 3. |
| Center/corners blocked | Use exact central cell and literal corners where reachable. Otherwise choose the nearest reachable central/corner cell deterministically, log it, and document. Confirm this fallback interpretation; never carve new passages. |
| All pacgums | Require both ordinary and power-pellet sets empty to clear. The wording could mean ordinary dots only; keep this predicate isolated for clarification. |
| Missing vs malformed config | Missing/invalid field: warn and continue using defaults as required. Unreadable or syntactically malformed file: clear CLI error and clean nonzero exit, because field recovery is not reliable. Subject does not explicitly require continuation for broken syntax. |
| Timeout and pause exit | Timeout ends game as a loss. Return to menu from pause abandons the session without submitting a score. Document both; timeout is explicitly left to the team, abandonment scoring is unspecified. |
| Name interpretation | Use ASCII A–Z, a–z, 0–9 and space, maximum 10; strip surrounding spaces; reject empty/space-only. Unicode alphanumeric acceptance and empty names are unspecified; document ASCII policy. |
| Timers and collisions | Power 6 s, eaten-ghost delay 5 s, respawn protection 1 s. Simulation-time timers; pickup before contact; loss before clear if both occur in one tick. These are choices, not subject constants. |
| Graphics/backend/platform | Prototype actual school-supported MLX Python binding first. No binding is supplied. Audit any substitute function by function. Release initially targets the tested development OS; universal cross-platform binaries are not required. |
| Dot count and levels | Place dots on every reachable non-spawn, non-power cell by default. The suggested pacgum count/key is not required. Normalize fewer than ten level entries by appending default level definitions; document this policy. |
| Optional work | Strict mypy, extra comment styles, smooth interpolation, richer visuals, distinct ghost personalities and additional cheats can wait. Sound/library features are not automatically allowed by the graphics restriction. |

## 1.6 Beginner preparation and proposed architecture

First practice Python functions, classes, lists, dictionaries, sets, exceptions, imports, virtual environments and basic Git. Then learn grid coordinates, binary masks, breadth-first search and finite-state machines from Chapter 2. You do not need advanced calculus, networking, a game engine, or machine learning. Ghost “AI” here is deterministic graph-based decision making.

```
pac-man.py                # argument boundary and application startup
pacman/
  contracts.py, config.py   # shared types; normalized settings
  maze_adapter.py, world.py # external maze -> validated game graph
  movement.py, rules.py     # player stepping; pickups and contact
  session.py              # levels, pause, clocks, results
  ghosts.py               # chase/flee/respawn decisions
  graphics.py, renderer.py # narrow MLX wrapper and scene drawing
  input.py, screens.py     # key -> intent; menus and text entry
  highscores.py           # validation and disk persistence
config.json, Makefile, README.md, .gitignore
packaging.spec            # or a packaging script at root
assets/                   # original/licensed sprites if used
project_management/       # decisions, schedule, actuals, risks, acceptance
```

This tree is a proposal for future implementation; this documentation task does not create those files. The simulation owns all game state. The renderer reads a snapshot, input creates commands, and storage saves a final result. The external package is isolated behind one adapter. This allows Developer A to work on rules while Developer B builds views and ghost policies using small fixture worlds.



### A real generated level

15 × 15 cells, seed 42
perfect=False, supplied wheel

Blue blocks: protected 42 motif
Yellow center: player start
Red corners: ghost spawns
Gold corner rings: power pellets
Small dots: ordinary pacgums

207 reachable cells
18 fully walled cells
202 ordinary dots + 4 power pellets

Walls define legal graph edges.
This drawing is an educational view,
not a prescribed artwork style.

*The rendered topology comes from the unchanged assigned generator, not a replacement.*

# Chapter 2 — Theoretical Foundations

Each section follows the same learning path: definition, motivation, mechanism, deeper reasoning, example, project use, mistakes, and further learning. Suggested algorithms are design choices unless the requirement register explicitly says otherwise. References are linked in §8.4.

## 2.1 Object-oriented design and typed boundaries

**A. Definition.** A class groups state and operations that preserve a rule, called an invariant. A protocol describes what a component must provide without exposing how it does it.

**B. Motivation.** If drawing code can also subtract lives or write score files, one key press can cause surprising effects. Narrow ownership makes changes local and tests independent of the window.

**C. Internal mechanism.** Define immutable settings and positions. Give GameSession ownership of score, lives and progression. Give a MazeGraph a neighbors(cell) method. Pass that interface to movement and ghosts instead of passing the graphics window.

**D. Deep technical explanation.** Composition means a session has a world, player and ghosts. It is usually simpler than a large inheritance tree. Static checking can reject a float where an integer cell is expected, but cannot prove that a cell is reachable: runtime validation still protects domain invariants. Keep raw JSON as object until validated; unchecked Any otherwise crosses every boundary.

### E. Practical example

```
from typing import Protocol
Cell = tuple[int, int]

class MazeGraph(Protocol):
    def neighbors(self, cell: Cell) -> tuple[Cell, ...]:
        """Return traversable neighboring cells."""
        ...
```

**F. Application to our project.** A01 freezes contracts; A03 implements the graph; A05 and B04 consume it. B02 consumes snapshots. A function docstring explains input meaning, return meaning and exceptions; annotations describe types.

**G. Common mistakes.** A global mutable dictionary defeats ownership. A class for every trivial value adds overhead without clarity. A successful mypy run does not validate files or prevent logical errors.

**H. Further learning.** Python dataclasses; typing.Protocol; PEP 257; mypy type narrowing. [S4, S7]

## 2.2 Configuration as a parsing and validation pipeline

**A. Definition.** Parsing turns text into generic values. Validation checks shape and meaning. Normalization converts incomplete or invalid values into safe, fully specified settings.

**B. Motivation.** The evaluator will edit configuration. A syntactically valid number may still cause a zero-size window or an impossible timer. Configuration must not leak partially checked values into the game.

**C. Internal mechanism.** Read UTF-8 with a context manager, remove whole comment lines, parse JSON, require an object, validate known fields, substitute or clamp defaults with specific warnings, and create typed settings. Keep the original file unchanged.

**D. Deep technical explanation.** Syntax, structure, type and range are different failure layers. Python bool is a subclass of int: isinstance(True, int) succeeds, so exact integer checks are needed for scores, dimensions and lives. NaN and infinity should not enter timers. Discard unknown keys instead of passing them as keyword arguments.

### E. Practical example

```
def strip_comments(text: str) -> str:
    """Remove full # comment lines while preserving line numbers."""
    return "\n".join(
        "" if line.lstrip().startswith("#") else line
        for line in text.splitlines()
    )

# '#1 player' inside a JSON string must remain unchanged.
# A field validator uses: type(value) is int, then range checks.
```

**F. Application to our project.** A02 implements R04–R05. Replacing comment lines with empty lines keeps parser line numbers useful. Leading whitespace before # is a supported extension. Inline # comments are not required.

**G. Common mistakes.** Splitting every line at # corrupts string values. Treating false as zero creates invalid settings. Silently defaulting all fields makes defense edits impossible to diagnose.

**H. Further learning.** Python json.JSONDecodeError; schema validation; pathlib; defensive parsing. [S1]

## 2.3 Grids, bit masks and undirected graphs

**A. Definition.** A grid cell has integer coordinates. Four low bits encode its north, east, south and west walls. A graph represents reachable cells as vertices and open passages as edges.

**B. Motivation.** A wall is a boundary between cells, not merely a tile marked solid. A single neighbors function should decide legality for the player, ghosts, reachability tests and pathfinding.

**C. Internal mechanism.** Use N=1, E=2, S=4, W=8. For a proposed direction, check target bounds, test the current wall and opposite target wall, and exclude fully walled cells. Cache adjacency after validation.

**D. Deep technical explanation.** For an undirected maze, E(x,y) must equal W(x+1,y); analogous symmetry holds north/south. A closed outer boundary prevents out-of-range moves. An integer 15 has all four bits set and is fully blocked. With V cells and at most four neighbors each, E is O(V), so storing adjacency costs O(V).

### E. Practical example

```
# 10 = 0b1010 = east + west walls; north/south are open.
mask = 10
can_go_north = (mask & 1) == 0   # True, subject to bounds
can_go_east = (mask & 2) == 0    # False
# Coordinates: cell (x=3, y=2) lives at walls[2][3].
```

**F. Application to our project.** The supplied wheel produces wall masks directly. A03 copies rows into immutable tuples and checks boundaries and symmetry; A04 uses the connected component for placement. Do not mistake the package’s shortest_path (entry to exit) for a ghost route to the moving player.

**G. Common mistakes.** Reversing row/column indices, treating a set bit as an opening, ignoring opposite walls, or placing dots in the blocked 42 motif produces impossible levels.

**H. Further learning.** Bitwise AND; adjacency lists; graph invariants; local wheel source [L2].

## 2.4 External generation, trees, cycles and adapters

**A. Definition.** An adapter translates an external interface into an internal contract. A perfect maze has one route between any pair of connected cells; an imperfect maze allows cycles.

**B. Motivation.** The subject assigns another team’s generator and reinstalls it at review. Editing it would hide integration problems and violate the rules. Cycles make chasing and evasion less constrained than a tree of dead ends.

**C. Internal mechanism.** Call the actual constructor with perfect=False. The supplied generator performs iterative depth-first carving, protects a central 42 pattern, adds passages and braids dead ends, then searches entry-to-exit. Your adapter only validates and translates its output.

**D. Deep technical explanation.** A connected tree has V−1 edges; adding an edge creates a cycle. DFS maintains a stack and visits available cells; BFS finds shortest unweighted routes. These explain the supplied package but are not instructions to replace it. Never depend on private methods or alter walls to repair a bad output.

### E. Practical example

```
from mazegenerator import MazeGenerator

raw = MazeGenerator(size=(15, 15), perfect=False, seed=42)
walls = tuple(tuple(row) for row in raw.maze)
# Validate walls, then expose your MazeGraph interface.
```

**F. Application to our project.** A03 is the only production module importing mazegenerator. A failed call becomes MazeLoadError with a helpful message. During a session, return to a usable menu; at startup, fail cleanly if the dependency is unavailable.

**G. Common mistakes.** Copying generator source into the project, modifying site-packages, calling generate() twice unnecessarily, or trusting the inaccurate quick-start constructor are avoidable failures.

**H. Further learning.** Adapter pattern; DFS spanning trees; cycle rank; wheel metadata and implementation [L2].

## 2.5 Breadth-first search and distance fields

**A. Definition.** BFS explores an unweighted graph in increasing distance from a source. A distance field stores the shortest number of edges from that source to every reachable cell.

**B. Motivation.** A ghost should not walk through a wall just because the player is geometrically close. Graph distance respects corridors. One search from the player can serve all four ghosts.

**C. Internal mechanism.** Put the source in a FIFO queue at distance zero. Remove the oldest cell, add each unseen neighbor at distance plus one, and repeat. To chase, choose a legal neighbor with the smallest distance; to flee, choose the largest.

**D. Deep technical explanation.** Every first discovery is shortest because all edges cost one and the queue processes whole distance layers. Time is O(V+E), memory O(V). Manhattan distance is a lower bound in a four-direction maze but ignores walls. Greedy fleeing increases immediate distance; it does not guarantee eventual escape, which is acceptable for the proposed behavior.

### E. Practical example

```
from collections import deque

def distances(graph: MazeGraph, source: Cell) -> dict[Cell, int]:
    """Compute shortest corridor distances from source."""
    dist = {source: 0}
    queue = deque([source])
    while queue:
        cell = queue.popleft()
        for nxt in graph.neighbors(cell):
            if nxt not in dist:
                dist[nxt] = dist[cell] + 1
                queue.append(nxt)
    return dist
```

**F. Application to our project.** B04 shares a field recomputed when the player changes cells. A04 uses BFS to prove pickups and spawns are reachable. Choose a stable tie order N,E,S,W for reproducible tests; varied per-ghost tie orders are optional.

**G. Common mistakes.** A list pop(0) shifts elements and makes queues slower. Mark visited when enqueueing, not after removal. Unreachable cells need a defined policy; do not compare missing distances as if they were zero.

**H. Further learning.** BFS shortest paths; collections.deque; Red Blob Games pathfinding explanations [S9].

## 2.6 Randomness and reproducible debugging

**A. Definition.** A pseudorandom generator evolves deterministic internal state. A seed selects the starting state, making a run reproducible when algorithm, calls and environment remain compatible.

**B. Motivation.** A fixed first maze supports teaching and review. Later random mazes add variety. Reproducing a bad level requires its dimensions, seed and package version, not just a screenshot.

**C. Internal mechanism.** Normalize first_seed to a positive integer (default 42). Use a private random.Random instance to choose positive seeds for later levels, then pass them to the external package. Log the actual seed with each level.

**D. Deep technical explanation.** The supplied wheel calls module-level random.seed; seed <= 0 invokes nondeterministic seeding. Surround its synchronous call with getstate/setstate in a finally block to avoid disturbing other users of module randomness. This is not concurrency isolation: keep generation single-threaded or isolate it in a process if later needed.

### E. Practical example

```
state = random.getstate()
try:
    raw = MazeGenerator(size=(w, h), perfect=False, seed=seed)
finally:
    random.setstate(state)
# Use a separate random.Random() for choosing later positive seeds.
```

**F. Application to our project.** A03 protects random state and records seeds. B04 can use stable tie breaks and require no randomness. The rule is random generation for later levels, not a guarantee that two random draws can never match.

**G. Common mistakes.** Passing seed=0 for the fixed level is not reproducible in this wheel. Random-sequence equality across every Python/package version is not promised. Do not mutate generator code to use a different RNG.

**H. Further learning.** random.Random; seed; getstate; setstate; deterministic replay [S2, L2].

## 2.7 Finite-state machines and transition ownership

**A. Definition.** A finite-state machine has named states and explicitly allowed transitions triggered by events. It represents rules such as “movement only happens while playing.”

**B. Motivation.** Separate boolean flags can allow contradictory conditions such as paused, victorious and playing at once. A single authoritative state makes input routing and testing clear.

**C. Internal mechanism.** Define MENU, HIGHSCORES, INSTRUCTIONS, PLAYING, PAUSED, VICTORY, GAME_OVER and NAME_ENTRY. Route input by state. Only GameSession produces level-completed or game-ended events; the screen controller maps those to views.

**D. Deep technical explanation.** Application state and ghost state are different machines. A ghost can be ACTIVE or EATEN_WAIT while the application is PAUSED. Power state is derived from a shared deadline, preventing four inconsistent ghost timers. Make end-session transitions idempotent: one result token can be saved only once.

### E. Practical example

```
MENU --start--> PLAYING --pause--> PAUSED
PAUSED --resume--> PLAYING
PAUSED --abandon--> MENU
PLAYING --last level clear--> VICTORY --continue--> NAME_ENTRY
PLAYING --no lives/timeout--> GAME_OVER --continue--> NAME_ENTRY
NAME_ENTRY --valid submit/save attempt--> MENU
```

**F. Application to our project.** A07 owns session transitions. B03 routes input and B06 renders terminal/name screens. A new Start resets all session state; a next level resets only level-local state.

**G. Common mistakes.** Letting a held Enter activate multiple screens can skip name entry. Consume transition input once and clear held movement intent when leaving gameplay.

**H. Further learning.** State pattern; transition tables; event handling; idempotency.

## 2.8 Game loops, fixed steps and pause-safe clocks

**A. Definition.** The game loop repeatedly processes input, updates rules and renders. A fixed simulation step advances rules by the same small duration regardless of rendering speed.

**B. Motivation.** Movement measured in pixels per frame becomes faster on a fast machine. Wall-clock deadlines continue during pause. Both errors break fairness and timing tests.

**C. Internal mechanism.** Measure elapsed real time with perf_counter, accumulate it only while PLAYING, and run update(dt) in fixed increments such as 1/60 second. Render each callback. While paused, refresh the real-time baseline without advancing simulation time; discard old accumulated time on resume.

**D. Deep technical explanation.** If speed is 5 cells/s, a 1/60-s tick adds 1/12 cell of movement credit. Every time credit reaches 1, commit one legal cell move. Timers use elapsed simulation seconds. A bounded catch-up budget prevents a stalled frame from freezing the UI, but dropping backlog slows simulation time relative to real time; document this choice.

### E. Practical example

```
# Pseudocode for one graphics loop callback
elapsed = now - last_real_time
last_real_time = now
if state == PLAYING:
    accumulator += min(elapsed, 0.25)
    steps = 0
    while accumulator >= dt and steps < 8:
        session.update(dt)
        accumulator -= dt
        steps += 1
    if steps == 8:
        accumulator = min(accumulator, dt)
else:
    accumulator = 0.0
render(snapshot)
```

**F. Application to our project.** A05 supplies movement credits; A07 owns simulation clocks; B01 connects the loop callback. Power duration, ghost delay, protection and level time all stop while paused. Unit tests pass dt directly and do not sleep.

**G. Common mistakes.** Using sleep for a ghost respawn blocks every event. Using time.time risks system clock jumps. A long gap on resume must not consume the level timer.

**H. Further learning.** time.perf_counter; fixed timestep; accumulator; simulation time [S3].

## 2.9 Movement, contacts and deterministic resolution

**A. Definition.** Movement turns intent into legal changes of position. Collision resolution determines what happens when entities occupy the same cell or cross the same edge.

**B. Motivation.** Testing only final cells misses a player and ghost exchanging positions in one tick. Testing one ghost at a time without a guard can remove four lives at once.

**C. Internal mechanism.** Store old positions. Choose proposed legal steps. Apply movement, collect a pickup once, resolve ghost contacts, then resolve death or level completion. A contact is equal new cells or exchanged old/new cells. Stop hostile processing after the first life loss.

**D. Deep technical explanation.** Use a simultaneous discrete-tick model: all actors propose moves from the same old state, then one rules pass commits outcomes. Limit actors to at most one grid step per tick or subdivide high speeds; otherwise swept paths, not just endpoints, are needed. Interpolation changes appearance only and must not change rules.

### E. Practical example

```
same_cell = player_new == ghost_new
edge_swap = (player_old == ghost_new and
             ghost_old == player_new)
contact = same_cell or edge_swap
# On contact: EATEN_WAIT -> ignore; edible -> eat; else lose one life.
```

**F. Application to our project.** A05 and A06 implement movement and contacts; B04 provides ghost proposals. With pickup-first ordering, entering a power-pellet cell shared with an active ghost makes that ghost edible during the contact pass.

**G. Common mistakes.** Teleport respawns should not be treated as a swept path across the maze. Remove eaten ghosts from contact handling during their wait. Respawn protection avoids immediate repeated deaths at the central spawn.

**H. Further learning.** Discrete simulation; collision ordering; swept collision; invariants.

## 2.10 Collections, scoring and idempotent events

**A. Definition.** Sets store unique pellet positions. An event is idempotent if processing it again cannot repeat its effect. Scoring should be tied to successful state changes.

**B. Motivation.** Rendering the player over a dot for ten frames must award points once. Eaten ghosts waiting to return must not award points again each frame.

**C. Internal mechanism.** If a position is in the dot set, remove it and add X. Otherwise check the power set, remove it, add Y and set power_until. For edible contact, transition the ghost to EATEN_WAIT before emitting +Z.

**D. Deep technical explanation.** Set membership/removal is average O(1). With non-negative rewards and no subtraction, score is monotonic. Integer arithmetic avoids rounding errors. The invariant score >= 0 is useful but not sufficient: tests also check exact deltas and event counts.

### E. Practical example

```
if player.cell in world.dots:
    world.dots.remove(player.cell)
    score += config.points_per_pacgum
# The next tick finds no dot, so no second reward is possible.
```

**F. Application to our project.** A04 creates disjoint sets; A06 owns all score changes; A07 carries score across levels. B06 cheats skip levels through the session transition API, not by manually adding arbitrary score.

**G. Common mistakes.** Placing both kinds of pellet on one cell, resetting score at each level, or accepting negative configured rewards breaks predictable scoring.

**H. Further learning.** Python set; idempotency; monotonic invariants; state-transition events.

## 2.11 Persistent storage and atomic replacement

**A. Definition.** Persistence keeps data after the process exits. Atomic replacement writes a complete new file and switches it into place instead of overwriting the only copy in place.

**B. Motivation.** Missing files, invalid JSON and unwritable locations are normal failures. A crash during a direct write can turn valid scores into a half-written document.

**C. Internal mechanism.** Load and validate each record; sort descending, keep ten. At save, create a temporary file in the destination directory, serialize validated rows, flush and close, then os.replace it onto the target. On error, remove the temporary file and keep the session alive.

**D. Deep technical explanation.** Same-filesystem replacement avoids readers observing a partially written destination. It does not make all failure modes disappear: durability across power loss can require fsync on the file and directory, and simultaneous writers need a separate lock. For this single-process game, atomic replacement plus clear errors is a sensible design.

### E. Practical example

```
rows = sorted(valid_rows, key=lambda r: r.score, reverse=True)[:10]
# Proposed JSON shape:
# {"version": 1, "entries": [{"name": "Ada 42", "score": 1200}]}
# Temp file beside target -> json.dump -> flush -> close -> os.replace
```

**F. Application to our project.** B05 implements load/save and validates type(score) is int, score >= 0. Stable sorting keeps existing entries first on ties. B06 reports a failed save and returns safely to the menu.

**G. Common mistakes.** Writing into an extracted one-file bundle may lose scores at exit. Trusting bool as an integer accepts true as a score. Catching every error and saying “saved” misleads the player.

**H. Further learning.** json; tempfile; os.replace; context managers; writable application data [S1, S5].

## 2.12 Graphics, input and resource ownership

**A. Definition.** A rendering backend creates a window, accepts events, provides an image buffer, presents that image and destroys its resources. A renderer maps a world snapshot to pixels.

**B. Motivation.** The subject restricts the graphical functions you use. A convenient high-level engine may provide physics, UI or other functions with no MLX equivalent. A narrow backend makes the compliance audit manageable.

**C. Internal mechanism.** Create one window and reusable image. Convert each cell to pixel coordinates, draw walls, pellets and entities, draw HUD text, then present. Register key/window-close hooks and a loop callback. On close, release images before the window and display resources as supported by the binding.

**D. Deep technical explanation.** Pixel location is y*stride + x*(bits_per_pixel/8), not always (y*width+x)*4. Stride and byte order belong to the backend. Draw custom circles or rectangles through a permitted pixel/image primitive rather than assuming an external shape helper is approved. Event callbacks should queue intent, not block on file dialogs or long operations.

### E. Practical example

```
px = origin_x + cell_x * tile_size + tile_size // 2
py = origin_y + cell_y * tile_size + tile_size // 2
# Reserve HUD height before computing maze fit.
# Graphics adapter: create_window / new_image / present / hooks / close.
```

**F. Application to our project.** B01 proves actual binding calls and MLX equivalents; B02 draws; B03 translates events. All four HUD fields remain visible during gameplay, including under a pause overlay.

**G. Common mistakes.** Allocating new images every frame leaks resources. Drawing outside the image corrupts memory in low-level bindings. A helper function written by your team is not automatically an impermissible external API, but its underlying library calls still need auditing.

**H. Further learning.** Official MiniLibX header; image buffers; event loops; actual binding docs [S6].

## 2.13 Packaging, dependency isolation and reproducibility

**A. Definition.** A development environment installs dependencies for coding. A distributable packages the application and required runtime resources so another machine can launch it.

**B. Motivation.** A program that only works from your repository directory is not a finished platform release. Native graphical libraries, assets and writable score storage are common differences between developer and player environments.

**C. Internal mechanism.** Create a clean venv, install the unchanged assigned wheel and graphics dependency, build a one-folder package from a root script/spec, include assets and a default config, and launch it from a fresh directory using packaged instructions.

**D. Deep technical explanation.** Source paths, bundled resource paths and writable data paths are distinct. A packaged launcher can supply its bundled default configuration while the source CLI still enforces exactly one user argument. Native binaries generally require a build/test for each supported OS and architecture; record the actual target rather than promising every platform.

### E. Practical example

```
# Development source launch:
python3 pac-man.py config.json
# Proposed packaged launch (through a documented launcher):
./pacman ./config.json
# Root packaging spec includes config, assets and native backend dependencies.
```

**F. Application to our project.** B08 owns a reproducible artifact and platform demonstration. Start with a one-folder build so missing assets and libraries are visible. Test that scores survive relaunch outside the source tree. [S10, S11]

**G. Common mistakes.** Uploading source alone does not prove an installable game. A platform page in draft mode can prevent reviewers downloading. Testing only on a machine with your environment hides missing dependencies.

**H. Further learning.** PyInstaller spec files; bundled resources; target OS; itch.io visibility and download access [S10–S12].

## 2.14 Testing, invariants and failure boundaries

**A. Definition.** A unit test isolates a small behavior. Integration tests check contracts between modules. Acceptance tests demonstrate subject requirements from the user’s perspective.

**B. Motivation.** Random games are hard to debug by playing alone. Tiny deterministic fixtures expose edge cases without waiting for a rare maze or collision.

**C. Internal mechanism.** Inject a clock, graph and score-store fake. Test individual rules first, then the real wheel and graphical backend, then a downloaded package. Keep a requirement-to-test matrix with evidence and observed results.

**D. Deep technical explanation.** An invariant must remain true after every valid transition: entities remain reachable, lives never become negative, score never decreases, and each session ends once. Property tests

can explore many seeds, but targeted examples still explain why failures matter. Boundaries need explicit failures: decoder error, generator exception, graphics initialization failure and disk permission error.

### E. Practical example

```
before = session.score
session.update(1 / 60)
assert session.score >= before
assert session.lives >= 0
assert session.player.cell in world.reachable
# Separate example: move two actors across one edge and assert one contact.
```

**F. Application to our project.** B07 curates acceptance runs; both owners write local tests. The subject says test programs are not submitted or graded; retain them in a local QA workspace and commit the required acceptance plan/results as management evidence.

**G. Common mistakes.** A screenshot cannot prove persistence, timing or reproducibility. Catching programming errors silently can make tests falsely pass; debug logs should preserve detail while production messages avoid tracebacks.

**H. Further learning.** unittest; pytest; test doubles; fault injection; property-based tests; boundary-value analysis.

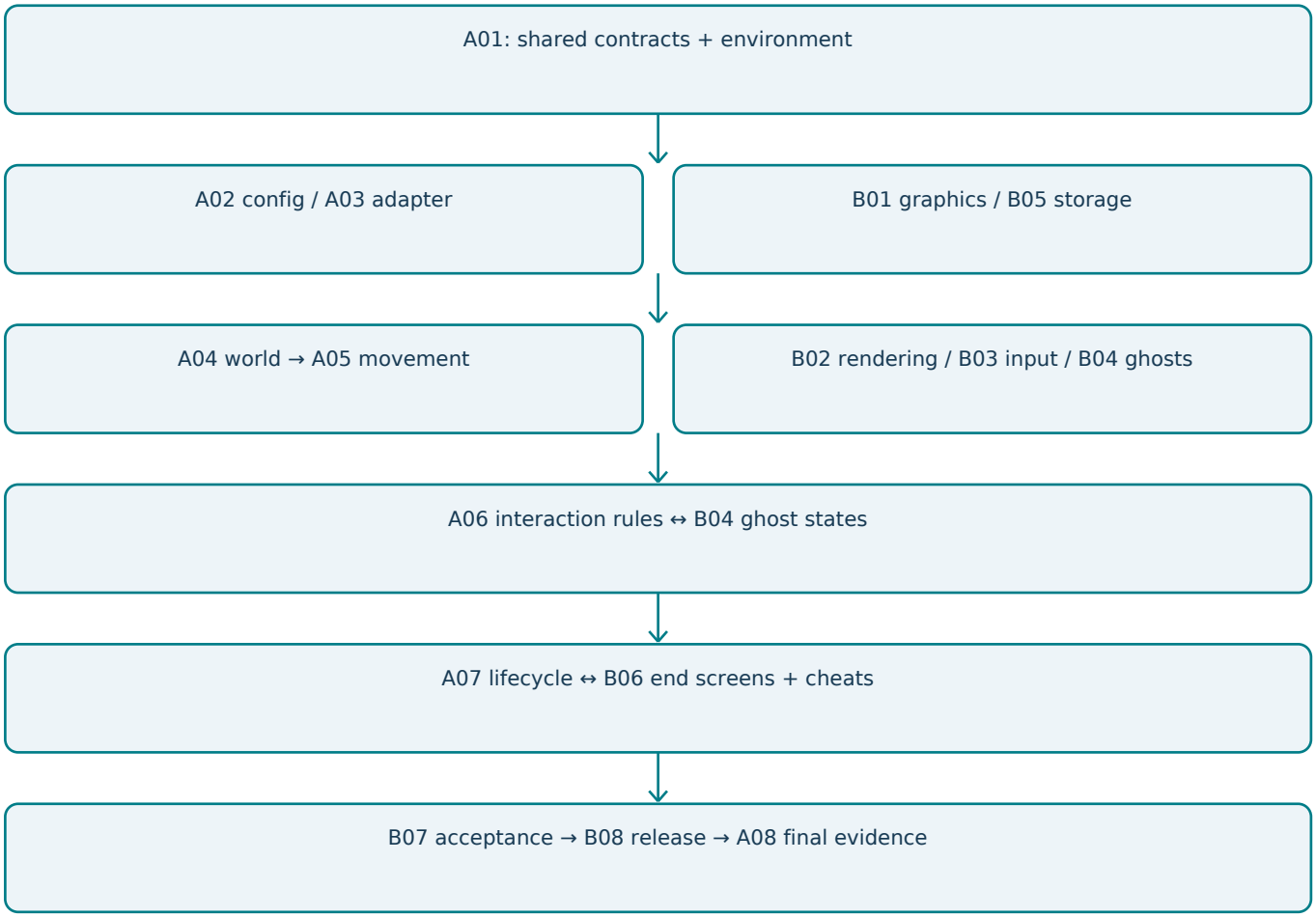
# Chapter 3 — Work Distribution

Exactly two people own the work. Each task has one accountable owner; the other developer reviews the interface and tests. Effort includes research, implementation, debugging, review and verification. These are planning estimates for beginners, not subject deadlines. Each person has 108 focused hours before contingency, with an uncertainty of roughly ±30%.

| Task ID | Task name | Owner | Difficulty | Hours | Dependencies |
|---|---|---|---|---|---|
| A01 | Contracts, environment and quality gate | A | Medium | 10 | None |
| A02 | CLI and robust configuration | A | Medium | 12 | A01 |
| A03 | External maze adapter and validation | A | High | 16 | A01 |
| A04 | Level assembly and reachable placement | A | High | 12 | A02,A03 |
| A05 | Player movement and simulation stepping | A | High | 16 | A01; A04 for live maze |
| A06 | Pickups, contacts, lives and score | A | High | 18 | A04,A05; B04 contract |
| A07 | Progression, clocks and session lifecycle | A | High | 14 | A02,A06; B03 contract |
| A08 | Management, README and defense evidence | A | Medium | 10 | A01; final inputs all tasks |
| B01 | Graphics feasibility and lifecycle | B | High | 14 | A01 contract |
| B02 | World renderer and HUD | B | High | 14 | B01; A04 snapshot |
| B03 | Input routing, menus and pause views | B | Medium | 12 | B01; A01 states |
| B04 | Ghost navigation and timed states | B | High | 20 | A01; A03 graph |
| B05 | Persistent top-ten repository | B | Medium | 12 | A01; A02 path policy |
| B06 | End screens, name entry and cheats | B | Medium | 12 | B03,B05; A06,A07 APIs |
| B07 | Acceptance harness and failure audit | B | High | 12 | Fixtures early; all core tasks final |
| B08 | Packaging and platform delivery | B | High | 12 | B01 spike early; all core tasks final |

Developer A owns configuration, external-world integration and the deterministic simulation. Developer B owns graphics/input, ghost decision-making, storage and release. Assigning ghosts to B balances A’s heavier collision and lifecycle work. The ghost module proposes legal moves; A’s rules module remains the single authority for resolving contacts and score. This avoids two competing engines.

## 3.1 Dependencies and parallel work




Arrows show integration gates, not an instruction to wait idle. B04 can use a hand-written graph fixture before A03 is ready; this is a test fixture, not a replacement maze generator. B02 can draw a frozen snapshot while A builds live worlds. A05 can test against a fixture before A04. A08 tracking begins on day one and B08 performs an early minimal-window packaging spike before the final build.

## 3.2 Shared work and balance

| Shared activity | Accountability and time allocation |
|---|---|
| Architecture agreement | A01 owner A; B contributes contract review within B01. Agree cell coordinates, units, ownership, states, errors and snapshots before broad implementation. |
| Rule/ghost integration | A06 owner A and B04 owner B pair on contact ordering. Each task estimate includes that developer’s participation; do not add a third “shared owner.” |
| Daily integration and reviews | Each owner budgets approximately 20% of their listed hours for review/testing. A reviews B’s release/storage; B reviews A’s config/maze/rules. |
| Final review and defense | B07 owns acceptance evidence, A08 owns assembled documentation, B08 owns the release. Each developer explains the other person’s modules. |
| Rebalancing checkpoint | At the first playable slice, compare actual hours and remaining work. Move rendering polish or documentation support if either person is >20% ahead, but preserve one accountable owner per task. |

## 3.3 Contracts to freeze before parallel coding

```
Cell = tuple[int, int]             # x, y; never pixel coordinates
MazeGraph.neighbors(cell)          # tuple of legal neighboring cells
MazeAdapter.generate(spec, seed)   # MazeData or MazeLoadError
WorldBuilder.build(maze, spec)     # immutable topology + mutable pellet sets
GhostPolicy.propose(snapshot, dt)  # proposed steps; no score/file changes
Rules.resolve(old, proposed)      # pickups, contacts, lives, score events
GameSession.update(dt, commands)  # sole mutation boundary for gameplay
GameSession.snapshot()            # read-only render data, no backend handles
ScoreStore.load()                 # validated entries + warnings
ScoreStore.save(entries)          # result with success/error, never false success
Renderer.draw(snapshot, ui)       # pixels only
```

Agree that time values use seconds and movement speed uses cells/second. A ghost carries its spawn corner, current cell and waiting deadline. Snapshots include application state, dimensions, walls, pellets, entities, score, lives, level, remaining time and visible cheat flags. A result token identifies one completed session and prevents duplicate score submission.

# Chapter 4 — Detailed Implementation Tasks

Execute these tasks against the interfaces in §3.3. File names are proposed future project files. Every task below has one owner, a concrete deliverable, edge cases and a measurable completion gate. Commands are examples to run during implementation, not claims that those files already exist. Use the active virtual environment for all Python tooling.

## Task A01 — Contracts, environment and quality gate

**Assigned to:** Person 1 — Developer A

**Objective:** Create a reproducible Python 3.10+ development skeleton and freeze the interfaces needed for parallel work. Covers R01–R03.

**Why this task matters:** A shared vocabulary prevents Developer A and B from implementing incompatible coordinates, units, errors and state transitions.

**Prerequisites:** Read Chapter 1 with B. Understand functions, imports, typing and context managers (§2.1 and §2.14). No project source is currently present.

**Deep theory:** Dependencies should point inward: platform and storage code depend on domain contracts, while movement rules depend on neither. A Makefile is a repeatable command interface; type checking catches interface drift before gameplay exposes it.

### Step-by-step implementation

**1. Prepare an isolated interpreter.** Check python3 --version. Create .venv with python3 -m venv .venv; activate it, then install flake8, mypy, a test runner and the local assigned wheel. Let B01 select the graphical binding. Record versions that actually work. *Why:* An isolated environment prevents accidental reliance on unrelated global packages.

**2. Define the module skeleton.** Create pac-man.py and pacman/__init__.py plus contracts.py. Put Cell, Direction, ApplicationState, configuration dataclasses, render snapshot and domain exceptions here. Write signatures before implementations. *Why:* A small importable package avoids importing the hyphenated CLI file from game modules.

**3. Agree invariants with B.** Record x/y ordering, seconds, cells/second, score integer rules, a read-only snapshot and who owns each transition. Add one tiny fixture maze in the local QA workspace. *Why:* Both streams can write independent implementations against a stable contract.

**4. Implement automation.** Create install, run, debug, clean and lint targets. Use the exact required lint commands shown below; debug invokes pdb. Keep clean limited to known build/cache paths, never config, assets or highscores. *Why:* Reviewers can reproduce normal operations without learning your shell history.

**5. Set quality conventions.** Add .gitignore for .venv, bytecode, caches and build artifacts. Add typed signatures and PEP 257 docstrings as code is written. Configure excludes for third-party/venv directories without weakening checks on owned code. *Why:* The supplied wheel is installed unchanged, not lint-repaired as project source.

**6. Verify the skeleton.** Run every Makefile target in a fresh environment. Confirm run/debug use a config argument and lint exits nonzero for an intentionally untyped local probe, then remove the probe. *Why:* A quality gate must detect failures rather than just appear to run.

### Practical example

```
# Makefile recipes below must begin with a TAB, not spaces.
.PHONY: install run debug clean lint
run:
    python3 pac-man.py config.json
debug:
    python3 -m pdb pac-man.py config.json
lint:
    flake8 .
    mypy . --warn-return-any --warn-unused-ignores \
      --ignore-missing-imports --disallow-untyped-defs --check-untyped-defs
# install: install pinned project dependencies, including the assigned wheel.
# clean: delete only explicitly listed caches/build outputs.
```

**Expected output:** Runnable package skeleton, documented contracts, dependency records, .gitignore and required Makefile targets.

### Definition of done

- ☐ Python version is at least 3.10 and recorded.
- ☐ All five mandatory targets exist and execute their intended operation.
- ☐ Owned functions/classes are documented and typed; the required lint commands pass.
- ☐ B approves the interface and can import contracts without a window.

**Common mistakes:** A Makefile recipe with spaces fails. Installing the wheel from an unrelated public package name risks using the wrong assignment. Avoid globally suppressing mypy errors or importing graphics in contracts.py.

**Integration:** B starts B01/B05 against these contracts. A02/A03 use settings and exception types. A08 links the contract decision record in management evidence.

## Task A02 — CLI and robust configuration

**Assigned to:** Person 1 — Developer A

**Objective:** Accept exactly one JSON path and turn comment-bearing input into bounded, typed settings with actionable warnings. Covers R04–R05.

**Why this task matters:** This is an explicit defense surface: reviewers will change values. Invalid data must never become a crash later in the renderer or generator.

**Prerequisites:** A01 contracts and §2.2. Coordinate schema/default choices with B05 and A07.

**Deep theory:** Separate syntax errors from semantic recovery. Pure field validators accept object and return typed values plus diagnostics. Validate all levels before opening a window, so dimensions cannot grow beyond display/resource budgets.

### Step-by-step implementation

**1. Define and document a schema.** Use highscore_filename, levels, lives, points_per_pacgum, points_per_super_pacgum, points_per_ghost and seed. Each level has width, height and level_max_time. Propose 10 default 15×15 levels, 3 lives, rewards 10/50/200, seed 42, 90 s. *Why:* These names/defaults are choices compatible with the indicative subject keys, not an extra required schema.

**2. Set explicit safe bounds.** As an initial tested policy use integer dimensions 7–31, lives 1–99, time 10–3600 s, points 0–100000 and positive seed up to 2^31−1. Use defaults for wrong types; clamp out-of-range integers and warn. Verify maximum dimensions fit B02's viewport. *Why:* Bounds limit invalid states and resource use; adjust only after tests and documentation agree.

**3. Build the CLI boundary.** In main(argv), require len(argv)==2 and a .json suffix (document case handling). Open with UTF-8 in a with block. Catch OSError, UnicodeError and JSONDecodeError; print a short diagnostic and return nonzero. *Why:* Unreadable/syntactically broken input cannot reliably be normalized field by field.

**4. Parse full-line comments.** Replace lines whose lstrip starts with # by blank lines, then call json.loads. Require a top-level object; document an error-and-exit policy for a wrong top-level shape. Ignore unknown keys. *Why:* Whole-line processing preserves hashes inside strings and parser line locations.

**5. Normalize nested levels and paths.** Missing/bad fields get defaults and key-specific warnings. Replace malformed level entries. Append defaults to reach ten; cap excessive list length with a documented resource limit such as 100. Resolve relative score paths against the config directory. *Why:* At least ten levels remains true even after evaluator edits; path behavior does not depend on launch directory.

**6. Exercise failure cases.** Test zero/two arguments, missing file, malformed JSON, whitespace comments, string hashes, booleans, floats, null, unknown keys and a short level list. Check terminal messages and normalized output without creating a window. *Why:* Fast parser tests isolate errors from unrelated graphical setup.

## Practical example

```
# Proposed config.json. Missing fields use documented defaults.
{
  "seed": 42,
  "lives": 3,
  "points_per_pacgum": 10,
  "points_per_super_pacgum": 50,
  "points_per_ghost": 200,
  "highscore_filename": "data/highscores.json",
  "levels": [
    {"width": 15, "height": 15, "level_max_time": 90}
  ]
}
# This example is padded to ten levels by the proposed normalization policy.
# Warning example: levels[0].width: expected integer; using 15.
```

**Expected output:** config.py, CLI validation, typed normalized settings and a README-ready defaults table. Include a complete shipped config containing ten explicit levels.

## Definition of done

- ☐ Every missing/invalid field has a documented recovery policy and useful warning.
- ☐ Unknown keys and full-line # comments do not cause failure.
- ☐ Every successful load has at least ten levels and non-negative rewards.
- ☐ No handled config error prints a Python traceback.

**Common mistakes:** Do not use bool(value) to parse intended numeric fields, truncate floats silently, or default the entire config because one field is invalid. A renamed key is a compatibility decision, not an automatic interpretation.

**Integration:** A04 consumes normalized level specs; A07 consumes timing/lives; B05 receives an already resolved score path. B06 instructions and A08 README use the same defaults.

# Task A03 — External maze adapter and validation

**Assigned to:** Person 1 — Developer A

**Objective:** Use the supplied wheel unchanged, hide its interface behind MazeAdapter, and return a validated immutable graph. Covers R06 and the seed portion of R07.

**Why this task matters:** The generator will be reinstalled during peer review. A clear boundary protects the game from interface mistakes, bad output and package exceptions.

**Prerequisites:** A01; §2.3–2.6. Inspect the actual wheel code and record the signature, coordinate ordering and wall masks.

**Deep theory:** Validation is a proof obligation: dimensions, integer masks, closed boundaries, reciprocal walls and connected playable cells must agree before gameplay. A connected-component traversal is O(V+E); it is validation, not generation.

## Step-by-step implementation

**1. Install and prove the real API.** Install the local wheel with pip in the venv. Import MazeGenerator and call size=(15,15), perfect=False, seed=42. Read maze after construction. Do not follow the incorrect width/height quick-start. *Why:* The supplied implementation is the only reliable integration target observed here.

**2. Write the wrapper.** In maze_adapter.py, call only the public constructor/property. Copy the grid to tuple-of-tuples so the rest of the application cannot accidentally mutate the package's state. Wrap import/generation failures in MazeLoadError with original details available to debug logs. *Why:* Other modules receive stable data and never need package-specific knowledge.

**3. Isolate random side effects.** Save global random state and restore it in finally around synchronous generation. Normalize a positive fixed first seed; use a separate random.Random instance to obtain positive later seeds. Record actual seed and dimensions. *Why:* The wheel reseeds Python's module RNG; restoring state avoids hidden coupling.

**4. Validate topology.** Require expected height and row width, exact int masks 0..15, closed exterior and matching opposite walls. Treat mask 15 as blocked. BFS from a valid corner or available cell; verify all intended walkable cells belong to the chosen playable component. *Why:* Malformed topology otherwise becomes out-of-range movement or unreachable pellets.

**5. Build neighbors and failure behavior.** Expose precomputed legal neighbors. Reject invalid output and show a clear menu error. A bounded retry with a logged new seed is an optional later-level recovery; never silently carve or substitute your own maze. *Why:* Clean rejection satisfies robustness without violating the external-generator constraint.

**6. Verify reinstall behavior.** Run default and boundary sizes across multiple seeds; compare repeat seed 42 runs. Force constructor failure with a fake provider. Reinstall the original wheel and rerun adapter tests. *Why:* Review conditions must not depend on a locally patched dependency.

## Practical example

```
DIRECTIONS = (
    (0, -1, 1, 4),  # north: dx, dy, wall, opposite wall
    (1, 0, 2, 8),   # east
    (0, 1, 4, 1),   # south
    (-1, 0, 8, 2),  # west
)
# For every cell (x,y) and each direction:
# 1. Check whether neighbor is in bounds.
# 2. If outside, current wall must be closed.
# 3. If inside, both wall bits must agree.
# 4. If both bits are open, include the neighbor in adjacency.
```

**Expected output:** MazeAdapter, MazeData/graph implementation, typed MazeLoadError, reproducibility log and integration evidence.

### Definition of done

• ☐ All real calls set perfect=False; package contents remain unchanged.

• ☐ Same positive first seed/dimensions yield identical topology in the tested environment.

• ☐ Malformed masks, boundaries and asymmetry are rejected clearly.

• ☐ The original wheel can be reinstalled without code changes.

**Common mistakes:** Seed 0 is nondeterministic in this package. shortest_path only connects package entry and exit. The fully walled 42 motif is intentional; do not fill it with dots or “repair” it.

**Integration:** A04 receives topology and reachable cells. B04 receives only neighbors, so it can work before this task completes. B08 includes the dependency in release builds.

## Task A04 — Level assembly and reachable placement

**Assigned to:** Person 1 — Developer A

**Objective:** Turn a valid maze into a completable level with a central player, four corner ghost spawns, four power pellets and dots in most corridors. Covers R07.

**Why this task matters:** A generated maze is geometry, not a finished level. Placement must make every collectible obtainable and preserve recognizable spawn rules.

**Prerequisites:** A02/A03; sets, graph reachability and deterministic ordering (§2.3, §2.5, §2.10).

**Deep theory:** A world combines immutable topology with mutable entity/collectible state. Store dots and power pellets as disjoint sets. Spawn selection is a constrained nearest-cell problem: minimize Manhattan distance to the desired anchor among reachable candidates, then break ties consistently.

### Step-by-step implementation

**1. Choose the playable component.** Use the adapter’s validated reachable set. Require a usable middle and four corner anchors, resolving only documented blocked-cell fallbacks. If distinct valid corner spawns cannot be found, reject cleanly. *Why:* A level should not begin with unreachable goals or ambiguous overlapping corner assignments.

**2. Define center and corners.** Use center=(width//2,height//2) and anchors (0,0),(w−1,0),(0,h−1),(w−1,h−1). For an unavailable anchor, choose nearest reachable candidate ordered by (distance,y,x), excluding already assigned distinct spawns. Log fallback use. *Why:* Even dimensions and ties now have deterministic, explainable behavior.

**3. Place entities and power pellets.** Place one ghost at each assigned corner and one power pellet there, keeping each ghost’s own spawn identity. Player starts at selected center. Exclude center from corner choices on degenerate input. *Why:* Respawning a ghost later requires its original corner, not just the nearest corner.

**4. Populate ordinary dots.** Use reachable cells minus player spawn minus power-pellet cells. This places dots in most corridors without relying on the optional suggested pacgum count. Keep the two pellet sets disjoint. *Why:* A set provides unique pickups and an explicit remaining-objective count.

**5. Create/reset a LevelState.** Store topology, spawn points, pellet sets, entities and level duration. Build a new instance for each level; do not mutate a shared configuration or fixture. *Why:* Fresh objects prevent a later level from inheriting already eaten pellets or ghost wait timers.

**6. Prove completion is possible.** Assert every pickup and spawn is reachable and total pellets is positive. Test the actual 15×15 default, rectangular maps, even dimensions and fixtures with blocked anchors. Render a debug overlay of chosen spawn cells with B02. *Why:* Topology tests catch unreachable goals before manual play wastes time.

### Practical example

```
# Default supplied-wheel observation: 207 reachable cells.
# With one center excluded and four power corners:
ordinary_count = 207 - 1 - 4   # 202 dots
power_count = 4
# At defaults, pellet-only points for this level:
pellet_points = 202 * 10 + 4 * 50   # 2220, before ghost rewards
# Count is derived from the observed map, not a subject-required total.
```

**Expected output:** world.py with WorldBuilder and LevelState, deterministic placements and map fixtures for integration.

### Definition of done

- ☐ Exactly four distinct ghost spawns and four power pellets exist.
- ☐ Player spawn follows documented middle policy.
- ☐ All pickups are reachable; power and ordinary sets do not overlap.
- ☐ No cells/walls of the supplied topology are modified.

**Common mistakes:** The geometric center can be a blocked cell in some layouts. Mutating the wall grid to make room violates the planned adapter boundary. Do not reuse mutable set defaults across instances.

**Integration:** A05 uses the graph/player spawn; A06 removes pellets; B02 draws this state; B04 uses each ghost's own spawn point; A07 calls the builder on transitions.

## Task A05 — Player movement and simulation stepping

**Assigned to:** Person 1 — Developer A

**Objective:** Implement four-direction, wall-respecting, speed-controlled movement with deterministic tick behavior. Covers the movement part of R08.

**Why this task matters:** Rules, ghosts and rendering need one trustworthy definition of position. Frame-dependent movement or illegal turns undermines every later feature.

**Prerequisites:** A01 contracts; A04 for real levels; §2.8–2.9. Agree input commands with B03.

**Deep theory:** Keep authoritative position on grid cells. Add movement credit speed*dt each tick; spend one credit on a legal step. Buffer a requested direction and try it before continuing the current direction. Reject blocked movement and discard excessive credit so a blocked player cannot accumulate a burst.

### Step-by-step implementation

**1. Define player movement state.** Store cell, facing direction, pending direction and movement credit. Start stopped at the center. Keep display interpolation separate if added later. *Why:* Integer cell positions simplify collision proofs and avoid tiny floating-point wall penetrations.

**2. Consume intent.** Accept normalized directions from B03, never raw key codes. Decide whether opposite direction is allowed immediately at the next cell step; document the chosen grid rule. *Why:* The model becomes independent of keyboard layout and graphical binding.

**3. Implement legal stepping.** When credit reaches one, try pending direction; if blocked try the current direction. Move only to a member of graph.neighbors(cell). If neither works, stay and cap/discard credit. *Why:* One adjacency function enforces identical wall rules everywhere.

**4. Bound per-tick travel.** Use dt=1/60 s with initial player speed 5 cells/s and validated maximum speed below 60 cells/s. For any future faster movement, subdivide updates rather than jumping over intermediate cells. *Why:* The simple collision and pickup rules assume at most one cell step per actor per tick.

**5. Expose old/new positions.** Return a movement proposal or preserve an old snapshot before committing. Do not change lives or score in movement.py. Reset movement credit on respawn and level change. *Why:* A06 must detect edge swaps and avoid inheriting stored velocity after a teleport.

**6. Test invariants.** Use a 3-cell corridor and an L-turn fixture. Verify blocked movement, buffered turns, arrows/WASD equivalence through integration, ten seconds of equal simulated time at different rendering rates, and pause with no movement. *Why:* Small fixtures reveal logic errors more clearly than a busy full maze.

### Practical example

```
# One tick, assuming speed * dt < 1:
credit += speed * dt
if credit >= 1.0:
    credit -= 1.0
    direction = legal_pending_or_current_direction()
    if direction is None:
        credit = 0.0
    else:
        cell = neighbor_in(direction)
# Rendering may interpolate; authoritative collisions still use grid cells.
```

**Expected output:** movement.py with typed player-step logic and deterministic movement fixtures.

### Definition of done

• □ Every committed step is a legal graph edge and exactly one cardinal direction.

• □ Elapsed simulation time, not render frame count, controls speed.

• □ Blocked/held inputs never cause a burst or wall crossing.

• □ Old/new cells and reset behavior are available to A06.

**Common mistakes:** Applying diagonal key combinations as a two-axis vector violates four directions. Keyboard repeat frequency should not determine speed. If interpolation is added, never round pixel coordinates back into game state.

**Integration:** B03 sends Direction commands; B02 draws committed position/facing; A06 resolves all side effects. A07 is the only owner of pause and dt delivery.

## Task A06 — Pickups, contacts, lives and score

**Assigned to:** Person 1 — Developer A

**Objective:** Make all scoring and collisions deterministic, prevent duplicate rewards, and implement safe life loss and central respawn. Covers R08 and R10.

**Why this task matters:** Contact rules are the highest-risk shared boundary: player and ghosts can move, cross, become edible, be eaten and respawn in one update.

**Prerequisites:** A04/A05 and B04’s proposal/state contract. Study §2.9–2.10 and agree priority with B before coding.

**Deep theory:** Treat a tick as a small transaction: gather proposals from old state, commit legal movement, apply pickups, then contacts, then emit death/clear events. A ghost’s transition to EATEN_WAIT is the scoring guard. A single life-loss flag bounds damage per tick.

### Step-by-step implementation

**1. Write the rule order first.** Record old positions, commit proposed legal moves, collect ordinary/power pellets, resolve contacts in stable ghost order, resolve one death, then report clear if still alive. A07 applies timeout before starting a tick whose time is already exhausted. *Why:* A documented total order makes simultaneous events testable rather than accidental.

**2. Implement one-shot collection.** Remove a dot before adding X. Remove a power pellet before adding Y and set power_until=sim_time+6. The proposed refresh rule replaces the deadline, not stacks duration. *Why:* Removal makes repeat processing harmless; a single deadline keeps ghost edibility consistent.

**3. Classify contacts.** Check equal destination cells and exchanged endpoints for each active ghost. Ignore EATEN_WAIT ghosts. If edible, transition to waiting and add Z once. If hostile and no protection/invincibility, mark life loss. *Why:* Both same-cell and crossing contacts are needed under discrete simultaneous movement.

**4. Resolve life loss.** Subtract one, bounded at zero. If lives remain, teleport the player to center, clear movement credit and grant 1 s simulation-time protection. Keep score, pellets and level time. Reset ghosts to corners and clear power as a documented respawn policy. *Why:* Protection/reset prevents immediate repeated losses while preserving meaningful level progress.

**5. Handle terminal and edge cases.** If no lives remain, emit a loss once and skip clear. If all pellets were eaten while alive, emit level complete. Zero reward values still consume pellets and trigger power. Disallow negative rewards in config. *Why:* The monotonic score and terminal-event invariants remain true even with unusual valid settings.

**6. Test contact tables with B.** Test hostile same-cell, hostile edge-swap, edible same-cell, edible edge-swap, four simultaneous hostile ghosts, power pickup plus contact, and the last dot plus fatal contact. Assert exact deltas and next states. *Why:* These cases exercise integration more effectively than long random play.

### Practical example

```
# At defaults: two dots + one power pellet + one eaten ghost.
score_delta = 2 * 10 + 50 + 200  # 270
# Expected events for an edible collision:
# GhostEaten(id=2), ScoreAdded(200)
# ghost[2].mode = EATEN_WAIT; ghost[2].ready_at = sim_time + 5
# Repeating the same contact while waiting adds no points.
# Four hostile contacts in one tick cost ONE life, not four.
```

**Expected output:** rules.py with authoritative pickup/contact resolution, life-loss handling and event fixtures.

### Definition of done

- □ Each pellet and each ghost-eating event awards exactly once.
- □ Score never decreases and lives never fall below zero.
- □ Crossing contacts are detected; multiple hostile contacts cost at most one life.
- □ Player respawns centrally with a documented reset policy; zero lives emits one loss.

**Common mistakes:** Mutating the ghost list while iterating can skip a contact. Testing power before pickup makes corner pellets unfair. Reusing movement old positions after teleport can create phantom edge swaps.

**Integration:** B04 supplies proposals and supports eaten/reset transitions; A07 receives clear/loss events; B02 gets updated snapshot state; B06 toggles cheat commands through the same rules boundary.

# Task A07 — Progression, clocks and session lifecycle

**Assigned to:** Person 1 — Developer A

**Objective:** Coordinate ten or more timed levels, pause/resume, score/life carryover and exactly-once terminal results. Covers R11 and the session side of R13.

**Why this task matters:** Individual rules are not a game until a session can start, progress, end, save and restart without stale state.

**Prerequisites:** A02/A06; B03 command contract; §2.7–2.8. Agree timeout and pause-abandon policy.

**Deep theory:** Separate SessionState (score, lives, level index, result token) from LevelState (topology, pellets, entities, elapsed time). A next-level transition replaces the second while preserving the first. Timers derive from active simulation time, so one pause gate freezes all temporal effects.

## Step-by-step implementation

**1. Build GameSession.** Accept normalized settings, a world factory and clock/tick dependencies. Start creates level index 0, score zero, configured/default 3 lives and fresh cheat flags. Load highscores through B05 at application startup. *Why:* Explicit construction makes restart behavior and tests repeatable.

**2. Own the update boundary.** Only update gameplay in PLAYING. Advance the level/session simulation clocks consistently; compare expiration before processing movement if the next tick would reach the time limit under the documented deadline-first rule. *Why:* Timing precedence must be consistent at the exact boundary.

**3. Implement pause/resume.** Transition to PAUSED on an edge-triggered command; keep rendering and processing menu input. Resume resets the real-time baseline and clears accumulator. Return to menu abandons the active session without saving under the proposed policy. *Why:* A paused screen remains responsive and cannot consume power or respawn delays.

**4. Advance levels.** After a successful clear, if another level exists, generate with a fresh random seed and reset level-local timers/entities/power while keeping score/lives. If the final level clears, create a victory result. *Why:* The required carryover applies between levels, not between distinct games.

**5. Handle terminal and generation failure.** Timeout produces loss. Zero lives produces loss. Store final score and a unique result token, freeze gameplay, and publish the result to B06. If generating the next maze fails, show a clear error and return safely to menu; do not fabricate a level. *Why:* A failed transition must not leave an old world pretending to be the next one.

**6. Verify complete runs.** Use a small graph and a next-level command to test all ten transitions, then play a real level without cheats. Check pause mid-power and mid-respawn, exact timeout, new-game reset, and a failing tenth-level factory. *Why:* Fixture-driven lifecycle tests cover rare paths quickly; real play proves the assembly works.

## Practical example

```
# Distinguish session and level reset:
# next level: preserve score/lives, reset time/pellets/power/entities
# new game:   score=0, lives=config.lives, index=0, cheats off

if level_cleared:
    if level_index + 1 == len(config.levels):
        finish_once(kind="victory", final_score=score)
    else:
        level_index += 1
        world = build_level(level_index, fresh_positive_seed())
```

**Expected output:** session.py with explicit transitions, testable clock logic and stable terminal-result contract.

### Definition of done

- □ At least ten levels are reachable; final completion wins.
- □ Score and remaining lives carry across levels.
- □ Pause freezes movement and every gameplay deadline.
- □ Timeout behavior is visible/documented and game-end result emitted exactly once.

**Common mistakes:** Resetting score/lives in the common level constructor erases progress. Using wall time for even one timer breaks pause. Holding a level-skip key must not jump through ten levels in one frame.

**Integration:** B03 owns views/input routing, B06 final/name UI, B04 ghost deadlines, B02 HUD. A08 documents the chosen ordering and reset table.

## Task A08 — Management, README and defense evidence

**Assigned to:** Person 1 — Developer A

**Objective:** Maintain real project evidence from day one, write the required English README, and prepare both developers to explain and modify the whole program. Covers R17–R19.

**Why this task matters:** The subject explicitly evaluates organization and understanding, not only game behavior. A retrospective fictional timeline is weaker evidence than short, honest updates recorded during work.

**Prerequisites:** A01 begins this task; final completion needs B07 acceptance results and B08 release evidence. Study Chapters 5–8.

**Deep theory:** Traceability links a requirement to an owner, implementation decision, test and outcome. A decision record captures context, options, choice and consequences, making future code changes explainable.

### Step-by-step implementation

**1. Create the evidence directory.** Create project_management/ with plan.md, actuals.md, decisions.md, risks.md, acceptance.md and retrospective.md, or equivalent clearly organized documents. Link it from README. *Why:* The directory is mandatory; these exact filenames are recommended formats.

**2. Track actual work.** Each workday record completed task IDs, hours used, remaining estimate, blocking issue and next action. Compare to the phase plan weekly. Document conflicts and how the two people resolved them. *Why:* Real tracking shows adaptation and supports workload rebalancing.

**3. Write the README skeleton early.** Use the exact required italicized first line with the two real logins. Add Description, Instructions, Resources, Configuration, Highscore, Maze Generation, Implementation, General Software Architecture and Project Management. *Why:* An early skeleton exposes missing contracts and prevents last-minute omissions.

**4. Fill implementation-specific instructions.** Document command-line usage, all defaults/bounds, # comments, score schema/path/ties, actual wheel API, graphics-call audit, controls/cheats, timeout/reset policies, package target and platform access. Include resources and precise AI-use disclosure. *Why:* A beginner or evaluator can reproduce the project without guessing hidden assumptions.

**5. Assemble evidence and review.** Record acceptance outcomes and unresolved defects honestly, link the packaging spec and tested release, and distinguish local non-submitted test programs from committed acceptance evidence. Check every R ID has an implementation/test reference. *Why:* Documentation should describe what exists, not a planned feature that never shipped.

**6. Rehearse cross-ownership defense.** A explains B04/B05/B08 while B explains A02/A03/A06. Practise a small reward change, HUD addition or name-rule adjustment, then rerun focused checks. *Why:* The subject

may request a live modification and expects responsibility for AI-assisted work.

### Practical example

```
# Exact README first-line template after replacing login placeholders:
*This project has been created as part of the 42 curriculum by login1, login2.*

# Decision-record example:
# D-04: Timeout ends the run as a loss.
# Reason: simple terminal behavior; no hidden score/life reset.
# Alternatives: restart level, subtract a life.
# Impact: A07 transition; B06 loss screen; test T11.
```

**Expected output:** English root README, ongoing project_management evidence, requirement traceability and a rehearsed defense checklist.

### Definition of done

- ☐ All mandatory README sections and exact first-line format are present.
- ☐ Actual progress, choices, risks and acceptance results are concrete and current.
- ☐ AI use is disclosed by task/part, and both developers can explain assisted code.
- ☐ Both developers can rebuild, demonstrate and make a small safe change.

**Common mistakes:** Do not present this guide as proof of work already done. Placeholder logins, stale defaults, broken links and a README that claims unsupported platform compatibility weaken review.

**Integration:** B supplies graphics, storage, cheat and release details. B07 owns measured acceptance results; A assembles them without replacing failures with assumptions.

## Task B01 — Graphics feasibility and lifecycle

**Assigned to:** Person 2 — Developer B

**Objective:** Choose and prove a Python graphics path compliant with the MLX-equivalent-function rule, including window creation, events, drawing and cleanup. Covers R15 and graphical parts of R02.

**Why this task matters:** Graphics feasibility is an early project risk. Discovering an unsupported native binding at packaging time can invalidate weeks of integration.

**Prerequisites:** A01 shared contracts; §2.12. No graphical library or binding is supplied, so its exact Python API remains to be established.

**Deep theory:** An abstraction is useful only when the implementation is real and tested. Keep the wrapper narrow: initialization, window/image creation, image memory access, presentation, event/loop hooks, text or owned bitmap text, destruction.

### Step-by-step implementation

**1. Identify the supported binding.** Check the school/development platform’s MLX Python binding and native prerequisites. Read its actual documentation/source and create a minimal import/window test. If using a substitute, audit every used function for an MLX equivalent. *Why:* “Similar library” is a function-level requirement, not permission to use an entire game engine.

**2. Record an API mapping.** List each wrapper operation, actual library call and MLX equivalent. Use initialization/window/image/presentation/hooks/destruction primitives. Implement custom geometry through those primitives if shape helpers have no equivalent. *Why:* The audit becomes concrete evidence for evaluation and limits accidental API drift.

**3. Build a minimal lifecycle.** In graphics.py initialize, create a window/image, draw one pixel or image, present, handle one key and close cleanly. Keep handles owned by the backend instance. *Why:* A tiny proof

verifies the hardest integration before game logic is attached.

**4. Connect a nonblocking loop.** Register callbacks that queue key/window events and call the application’s tick/render function. Use Python perf_counter for elapsed time; do not block with input(), sleep() or a second event loop. *Why:* The window must process events even while menus or error messages are visible.

**5. Add failure cleanup.** Catch initialization and asset errors at the boundary. Destroy resources in reverse creation order, accounting for partial initialization. Make close safe if called twice. *Why:* Errors during setup must not leak handles or produce a crash/traceback during review.

**6. Test and spike packaging.** Open/close repeatedly, resize only if supported, test focus loss and window close from each screen. Give B08 a minimal-window build early; document OS/native dependencies. *Why:* Packaging the backend early detects missing dynamic libraries and display assumptions.

### Practical example

```
# Abstract wrapper operations, NOT a claim about a Python binding API:
# initialize()          -> mlx_init
# create_window(...)    -> mlx_new_window
# create_image(...)     -> mlx_new_image / mlx_get_data_addr
# present(image)        -> mlx_put_image_to_window
# register events      -> mlx_hook / mlx_key_hook
# register frame       -> mlx_loop_hook
# destroy image/window -> mlx_destroy_image / mlx_destroy_window
# Check actual binding signatures and supported cleanup for the target OS.
```

**Expected output:** graphics.py backed by verified calls, graphics audit record, minimal-window demonstration and cleanup tests.

### Definition of done

- ☐ Every used graphical-library function has an identified equivalent.
- ☐ A window renders, accepts input and closes on the intended target.
- ☐ Partial initialization errors are handled with clear messages and cleanup.
- ☐ The backend can be packaged in a minimal smoke build.

**Common mistakes:** Do not invent a Python MLX API from the C names. Sound, font engines, physics and UI widgets are not automatically allowed. Never let a broad exception handler continue using invalid native handles.

**Integration:** B02 and B03 use the wrapper; A07 provides the tick callback. B08 records native dependencies and target environment; A08 links the audit.

# Task B02 — World renderer and HUD

**Assigned to:** Person 2 — Developer B

**Objective:** Render a readable maze, all entities/pellets and the four mandatory HUD values, using snapshots and permitted primitives. Covers the graphical view in R13.

**Why this task matters:** The player must understand walls, threats, power state and remaining objectives immediately. Visual clarity is more important than complex artwork.

**Prerequisites:** B01, A01 snapshots and A04 world data; §2.3 and §2.12.

**Deep theory:** Rendering is a coordinate transformation from grid space to screen space. Keep the simulation independent of pixel size. A stable layout computes available width/height after reserving HUD and margins, then chooses the minimum integer tile size that fits.

## Step-by-step implementation

**1. Design a readable layout.** Reserve a HUD band and margins. Compute tile size from maze dimensions and the target viewport. Require a tested minimum tile size and coordinate with A02 maximum dimensions. *Why:* A valid config should not silently create a clipped or unreadable window.

**2. Draw topology correctly.** Draw closed wall edges using masks, avoiding inconsistent wall thickness. Draw blocked cells distinctly. Implement circles/rectangles in owned raster code over the verified image buffer if needed. *Why:* Wall masks describe boundaries, and the picture must agree with movement.

**3. Draw pickups and actors.** Use small dots and larger corner pellets. Draw a visible player/facing direction, four ghosts, a clearly distinct edible appearance, waiting-state absence/indicator and respawn protection feedback. *Why:* State changes need visual feedback for both players and evaluators.

**4. Draw HUD and overlays.** Show score, lives, level number (human-facing index+1) and remaining time on every gameplay frame. Keep them readable under pause/cheat overlays; use consistent alignment and contrast. *Why:* The subject explicitly requires all four values always visible during gameplay.

**5. Reuse resources.** Allocate buffers and static assets once or at level/size changes, not per frame. Draw snapshots only; do not remove pellets or change score while painting. *Why:* Repeated renders of the same snapshot must not change game behavior or leak resources.

**6. Check extremes visually.** Render smallest/largest accepted dimensions, long score values, zero seconds, last life, all ghost states and pause. Compare wall drawings to graph neighbors on a small fixture. *Why:* Data tests cannot detect clipped text or invisible edible ghosts.

## Practical example

```
hud_height = 64
margin = 16
available_w = window_width - 2 * margin
available_h = window_height - hud_height - 2 * margin
tile = min(available_w // width, available_h // height)
origin_x = (window_width - tile * width) // 2
origin_y = hud_height + margin
# Reject/normalize configurations whose tested tile size is unreadable.
```

**Expected output:** renderer.py with consistent world/HUD drawing, documented sizing and reusable image resources.

## Definition of done

- ☐ Every wall/pickup/entity is visible and agrees with the model.
- ☐ Score, lives, level and time remain visible in gameplay and pause.
- ☐ Maximum accepted configuration fits the chosen viewport.

• □ Repeated rendering leaves the model unchanged and resource use stable.

**Common mistakes:** Using width in place of row stride can corrupt image memory. A power pellet hidden under a ghost needs a consistent draw order or visible halo. Avoid tiny fonts on maximum-sized maps.

**Integration:** A supplies snapshots; B03/B06 supply view/overlay state. B07 performs visual acceptance and records target display dimensions.

## Task B03 — Input routing, menus and pause views

**Assigned to:** Person 2 — Developer B

**Objective:** Build keyboard controls and all navigation screens, routing commands by application state. Covers menu/pause portions of R13 and input in R08.

**Why this task matters:** A playable game needs a complete route from launch to exit, not just a moving character. State-specific input prevents menu keys from accidentally controlling gameplay.

**Prerequisites:** B01 and A01 state/command contracts; §2.7. A07 owns pause mechanics while B owns the presentation.

**Deep theory:** Events report discrete presses/releases; held-key state represents continuous intent. Menus need edge-triggered activation, while movement should continue independently of OS key-repeat rate. Clear intent when focus is lost or gameplay exits.

### Step-by-step implementation

**1. Map input into commands.** Translate arrow keys and/or WASD into Direction; select a documented pause key such as P/Escape, Enter for selection and menu navigation keys. Handle key-release and window-close events. *Why:* The model never needs platform-specific key codes.

**2. Build the main menu.** Provide Start Game, View Highscores, Instructions and Exit. Indicate selected item and make every route usable by keyboard. Read scores from B05’s current validated list. *Why:* These are named mandatory options, not optional polish.

**3. Add instructions and score views.** Explain movement, collecting, power, lives, ten-level objective, timer policy, pause and cheat controls. Show top ten names/scores and a clear empty state. *Why:* Players and evaluators should not need the source code to discover controls.

**4. Build the pause overlay.** Offer Resume and Return to main menu. Send commands to A07 rather than freezing time locally. Keep HUD visible and show that the session is paused. *Why:* One timing owner prevents ghost timers from advancing behind the overlay.

**5. Debounce transitions.** Consume the activating press once. Clear held movement/selection state on screen changes and focus loss. Process a queued close in all states through B01 cleanup. *Why:* A held Enter should not start a game and instantly trigger a second action.

**6. Run navigation tests.** From launch visit every screen, return, start, pause, resume, abandon, start again and exit. Repeat with held keys and focus loss. *Why:* Navigation loops reveal stale state and missing return routes early.

### Practical example

```
# State-aware routing (pseudocode):
if state == PLAYING:
    send(movement_or_pause_command(event))
elif state == PAUSED:
    send(resume_or_abandon_command(event))
elif state == NAME_ENTRY:
    edit_name_buffer(event)
else:
    navigate_current_menu(event)
# Window-close is handled in every state before ordinary routing.
```

**Expected output:** input.py and screens.py with documented controls, main/instructions/highscore screens and pause overlay.

### Definition of done

• □ All four main menu choices work and return/exit correctly.

• □ Pause Resume and Return commands reach A07.

• □ Held keys and focus changes do not cause duplicate transitions/stuck motion.

• □ Controls and game rules are discoverable in the Instructions view.

**Common mistakes:** Never use blocking terminal input for menu/name entry. A pause overlay alone does not pause the simulation. Avoid passing characters typed as a name to movement handling.

**Integration:** A05 consumes directions; A07 consumes lifecycle commands; B05 supplies scores; B06 extends end/name/cheat input without duplicating menu logic.

## Task B04 — Ghost navigation and timed states

**Assigned to:** Person 2 — Developer B

**Objective:** Implement four autonomous ghosts that chase, flee when edible and reappear at their own corners after being eaten. Covers R09 and power behavior in R10.

**Why this task matters:** The subject requires meaningful pursuit and evasion but permits your chase algorithm. A simple graph-distance policy is explainable, testable and sufficient.

**Prerequisites:** A01 graph/state contracts, A03 for real topology, §2.5–2.9. Use fixture graphs until A's adapter is ready.

**Deep theory:** A BFS distance field from the player gives shortest corridor distances for all ghosts. Minimize neighbor distance for chase, maximize for flee. Ghost identity stores its own corner. ACTIVE/EATEN_WAIT plus the session power deadline determine behavior without inconsistent per-ghost power timers.

### Step-by-step implementation

**1. Define ghost data.** Create four records with id, spawn, cell, movement credit, state and ready_at. Use a tested default speed such as 4 cells/s, below the player's 5, and a 5 s eaten wait. *Why:* Independent identity and timers prevent every ghost from respawning in the same corner.

**2. Compute shared distances.** Use BFS from the current player cell and cache by player cell/topology. Recompute on player movement or level replacement. *Why:* One O(V+E) traversal serves all four ghosts rather than four repeated full searches.

**3. Choose legal directions.** For each movement-ready ACTIVE ghost, inspect legal graph neighbors. Chase takes minimum distance; edible mode takes maximum. Use deterministic ties and allow reversal when it is the best legal move. If no legal neighbor, stay and report/debug the invalid-world condition. *Why:* The policy respects walls and visibly changes behavior under power.

**4. Implement wait and return.** When A06 eats a ghost, set EATEN_WAIT and ready_at=sim_time+5, remove it from movement/contact/rendered active actors. At or after the deadline, place it at its stored corner and reset movement credit. *Why:* Waiting is a state with a deadline, not a blocking sleep.

**5. Coordinate power and freeze.** Read a single session power deadline; a returning ghost is edible if power remains active under this guide's policy. A freeze cheat stops movement but not other timers unless explicitly documented; pause stops all timers. *Why:* Two different controls should not accidentally share contradictory meanings.

**6. Test behavior and integration.** Use a corridor where chase decreases distance and flee increases it, a junction tie, a dead end, a loop, same-cell contact, edge swap and a return at 4.99/5.00 s. Pair with A06 on contact priorities. *Why:* Tests prove both the navigation decision and the lifecycle contract.

### Practical example

```
# Pseudocode after obtaining one BFS field d:
choices = graph.neighbors(ghost.cell)
if choices:
    if edible:
        next_cell = max(choices, key=lambda c: d[c])
    else:
        next_cell = min(choices, key=lambda c: d[c])
# neighbors has stable ordering, so ties are reproducible.
# If ghost.state == EATEN_WAIT: no step until sim_time >= ready_at.
```

**Expected output:** ghosts.py with shared-distance navigation, four persistent identities and nonblocking timed respawn.

### Definition of done

- ☐ All four ghosts move only along legal edges and chase/flee as specified.
- ☐ Eaten ghosts award no repeated contact/reward while waiting.
- ☐ Each returns to its own corner after the documented simulation delay.
- ☐ Pause freezes deadlines; cached distance fields refresh correctly.

**Common mistakes:** Maximizing Euclidean distance can move toward the player around a wall. A cached field from a previous level is invalid. Do not let ghost AI directly subtract lives or update persistent scores.

**Integration:** A06 commits contacts and score; A07 owns simulation time and resets; B02 visualizes edible/waiting states. Provide fixtures to B07 for reproducible tests.

## Task B05 — Persistent top-ten repository

**Assigned to:** Person 2 — Developer B

**Objective:** Validate, load, rank and safely save ten player scores, handling file problems without crashing. Covers the storage part of R12.

**Why this task matters:** Persistence must survive invalid files and relaunch, and must not claim success when disk writes fail.

**Prerequisites:** A01 data contracts, A02 resolved path policy and §2.11. Agree validation with B06's name entry.

**Deep theory:** Treat file contents as untrusted object values. Validation creates typed ScoreEntry records. Stable descending sort handles ties deterministically. Atomic replacement protects the previous file from partial normal writes; it does not solve concurrent-process merging.

### Step-by-step implementation

**1. Define schema and validation.** Use version=1 and entries containing name/score. Strip name edges, allow ASCII alphanumeric/space only, require 1–10 characters and exact int score >=0. Share the validator with B06. *Why:* One validator prevents the UI and file loader from accepting different data.

**2. Implement tolerant load.** Missing file gives empty scores. Invalid document/schema gives empty scores plus warning. For a valid entries list, skip invalid rows with diagnostics while keeping valid ones. Sort descending and keep ten; document stable tie order. *Why:* A damaged row need not discard valid history, and startup remains usable.

**3. Implement safe save.** Ensure the destination directory can be created, write a temporary file beside the target with a context manager, serialize validated top ten, flush/close and os.replace. Remove leftover temp files in failure cleanup. *Why:* The old leaderboard remains intact until the new version is complete.

**4. Return honest outcomes.** Return a typed success/error result with a user-safe message. Keep the just-finished result in memory on failure; never report “saved” unless replacement succeeds. Support one save attempt/retry flow per result token. *Why:* The UI can tell the truth and avoid inserting duplicates during retries.

**5. Document writable location.** Resolve relative configured paths against the config location in source mode. In packaged mode ship a launcher/config policy pointing to a writable user data location, not a temporary bundle extraction directory. *Why:* Persistence must survive different working directories and packaged restarts.

**6. Test corruption and permissions.** Use a temporary QA directory. Test missing file, truncated JSON, bad schema, booleans/floats/negative scores, illegal/long names, 11 entries, ties, unwritable directory and failure before replacement. Reload after a successful save. *Why:* A true persistence test reads from disk in a fresh store instance.

### Practical example

```
# Example validated leaderboard document:
{"version": 1, "entries": [
    {"name": "Ada 42", "score": 270},
    {"name": "Lin", "score": 120}
]}
# Valid: "A", "Ada 42", score=0
# Invalid: "", "          ", "Ada!", 11-character names
# Invalid scores: -1, 1.5, "100", true
```

**Expected output:** highscores.py with shared validation, robust read/write, stable top ten and documented storage policy.

### Definition of done

- □ Loading broken/missing files never crashes the game.
- □ Only valid names and non-negative integer scores survive loading/saving.
- □ At most ten entries persist, ordered highest first.
- □ Successful save survives a fresh process; failed save is reported clearly.

**Common mistakes:** Do not truncate a corrupt file before preparing the replacement. A permission test using only file mode bits may not fail when run as a privileged user; inject OSError as well. Avoid treating JSON true as score 1.

**Integration:** B03 displays loaded rows; B06 validates/submits names; A02 resolves config paths; B08 tests writable storage from the installed/downloaded artifact.

# Task B06 — End screens, name entry and review cheats

**Assigned to:** Person 2 — Developer B

**Objective:** Finish the required win/loss/name/menu loop and provide visible, useful evaluation controls. Covers terminal UI in R12–R13 and R14.

**Why this task matters:** Reviewers need to reach rare states quickly. Name entry and saving must work identically after both victory and loss.

**Prerequisites:** B03/B05, A06/A07 command/results interfaces; §2.7 and §2.11.

**Deep theory:** A result token represents one finished game. Name-entry state holds editable text and validation feedback but cannot change the final score. Cheats are explicit commands handled by the same session boundary as ordinary input, keeping transitions testable.

## Step-by-step implementation

**1. Build both result views.** Game Over displays final score; Victory displays final score and congratulatory text. Each continues to a name prompt or includes it directly. Disable gameplay mutation once a result exists. *Why:* The subject requires both endings, not only a generic loss screen.

**2. Implement nonblocking editing.** Keep a text buffer, append only allowed characters, support Backspace and Enter, enforce max length 10 and visibly explain invalid/empty input. Consume keys in NAME_ENTRY only. *Why:* Native terminal input would freeze the graphical event loop and lose window control.

**3. Submit exactly once.** Combine validated name and the immutable result score, merge/rank through B05, then return to menu. On failure show a warning and offer a retry/back route without duplicate insertion. *Why:* Repeated Enter or disk failures must not create duplicate leaderboard rows.

**4. Add useful cheat commands.** Propose F1 toggle cheat mode, F2 invincibility, F3 skip current level, F4 ghost freeze and F5 add a life up to a documented cap. Display active flags and list controls in Instructions/README. *Why:* The subject lists suggestions; this selected set makes loss/win, rendering and navigation easy to review.

**5. Define exact cheat semantics.** Skip invokes A07’s normal level-complete route and adds no pellet points. Freeze stops ghost movement while their deadlines continue; pause freezes all time. Invincibility prevents hostile life loss but still allows eating edible ghosts. Reset flags on new game. *Why:* Explicit semantics prevent cheats from bypassing the logic reviewers need to observe.

**6. Test complete user paths.** Reach loss with one life and victory by skipping nine levels then clearing/skipping the tenth. Enter max-length names, reject punctuation, backspace, save and relaunch. Test held cheat keys and toggles during pause. *Why:* The complete loop proves screen transitions and persistence together.

## Practical example

```
# Proposed controls (choices, not subject-mandated key bindings):
F1  toggle evaluation mode
F2  toggle invincibility (only when evaluation mode is enabled)
F3  complete current level once
F4  toggle ghost movement freeze
F5  add one life, bounded at 99
# Turning evaluation mode off clears its active modifiers.
# Display: CHEATS ON | INVINCIBLE | GHOSTS FROZEN
```

**Expected output:** Win/loss/name-entry screens, duplicate-safe save flow and documented visible cheat controls.

### Definition of done

- □ Both win and loss display final score and accept a valid name.
- □ After saving/handled failure, the menu is reachable and usable.
- □ Cheats can be activated and visibly help test core features.
- □ Held inputs do not repeatedly save, skip levels or add lives unintentionally.

**Common mistakes:** Cheat features are suggested, so do not falsely label all listed powers mandatory. Decide whether cheated scores are recorded; this guide records them like other valid scores because exclusion is not required, and documents that choice.

**Integration:** A07 processes lifecycle/cheat commands; A06 reads invincibility; B04 reads freeze; B05 owns persistence; B03 documents controls and routes state-specific input.

## Task B07 — Acceptance harness and failure audit

**Assigned to:** Person 2 — Developer B

**Objective:** Coordinate measurable acceptance coverage for every requirement and verify the integrated game handles realistic faults. Covers R02 and evaluation aspects of R19.

**Why this task matters:** Unit tests can all pass while a menu route, packaging asset or reviewer config edit still breaks the application. Acceptance checks close those gaps.

**Prerequisites:** Start fixtures after A01; final run needs all core tasks. Use Chapter 7 as the test plan.

**Deep theory:** Testing should cross boundaries deliberately: pure functions, adapter/library integration, graphical state transitions and a fresh installed artifact. Deterministic fixtures make failures reproducible; real-package and real-window runs catch false assumptions in those fixtures.

### Step-by-step implementation

**1. Build a local QA workspace.** Keep unit/integration test programs outside submitted deliverables as the subject requests. Store tiny graphs, corrupt config/score fixtures and a fake clock there. Commit the acceptance plan/results in project_management. *Why:* Test programs and graded management evidence have different submission roles.

**2. Map each requirement.** Use R01–R19 to enumerate tests and assign a producer for each piece of evidence. Capture expected result, observed result, environment, seed and pass/fail. *Why:* A green aggregate count cannot hide a requirement that was never exercised.

**3. Automate deterministic cases.** Exercise config normalization, maze validation, movement, contacts, ghost wait, pause and score storage. Use fake time and injected errors rather than sleeps or permanently changing the assigned wheel. *Why:* Fast, repeatable tests isolate logic and preserve the external dependency.

**4. Run real interaction sessions.** Execute menu loops, both endings, name entry, all HUD values, cheats, focus loss/window close and maximum dimensions with the actual backend. Reinstall the assigned wheel and rerun. *Why:* Input and native resources need real environmental verification.

**5. Audit failure boundaries.** Force unreadable config, invalid score JSON, failing generator, unavailable asset and unwritable score directory. Verify clear messages, cleanup and a usable menu or clean nonzero startup exit, with no user-facing traceback. *Why:* Robustness is observable behavior, not merely the presence of try/except.

**6. Retest focused fixes and gate release.** For each defect record reproduction, root cause, owner and confirming test. Run exact lint commands, complete acceptance and hand B08 a release candidate. After packaging, rerun the critical user journeys on the downloaded build. *Why:* Each stage validates what the

next stage depends on and prevents shipping known blockers.

### Practical example

```
# Local QA example; fixture helpers are test-only, not production generators.
def test_hostile_edge_swap(session_fixture):
    s = session_fixture(player=(1, 1), ghost=(2, 1), lives=3)
    s.propose(player=(2, 1), ghost=(1, 1))
    s.resolve()
    assert s.lives == 2
    assert s.player.cell == s.world.player_spawn
# Evidence row: T08 | R08/R10 | seed/fixture | expected | observed | owner.
```

**Expected output:** Executed traceability matrix, local QA suite, defect log and release readiness report.

### Definition of done

- ☐ Every R ID has a recorded acceptance outcome and evidence.
- ☐ Both endings, all screens and all major failure boundaries were exercised.
- ☐ Mandatory lint passes on owned source and no critical defects remain.
- ☐ B08 receives an identified source revision and repeatable smoke checklist.

**Common mistakes:** Do not report planned tests as passed. Failure injection must not alter the submitted package. A clean exit on missing configuration is acceptable under the documented policy; an unexplained traceback is not.

**Integration:** Both owners fix their modules. A08 curates the evidence, B08 tests the packaged artifact, and B07 verifies fixes against the same fixtures.

## Task B08 — Packaging and platform delivery

**Assigned to:** Person 2 — Developer B

**Objective:** Produce a fully working installable/launchable free unlisted/private release on a public gaming platform and keep its build reproducible. Covers R16 and packaging review in R19.

**Why this task matters:** The subject requires a real demonstrated platform build, not only a local executable. Review may require regenerating it.

**Prerequisites:** B01 early packaging spike; final release requires B07 core acceptance and A08 instructions. Read §2.13 and verified platform documentation.

**Deep theory:** A build artifact captures code, dependencies and resources for a target environment. Reproducibility requires the root spec/script and recorded versions. Resource paths must be independent of the working directory, while highscores must be stored somewhere persistent and writable.

### Step-by-step implementation

**1. Choose a realistic target.** Start with the OS/architecture on which the MLX binding works. Use a one-folder PyInstaller-style build if compatible, or an equivalent tested packaging method. Record target, Python and dependency versions. *Why:* A single tested target is more credible than untested cross-platform claims.

**2. Create a root build recipe.** Add packaging.spec or a root script that includes the source, assigned wheel dependency, assets, default config and native graphics libraries. Keep source CLI semantics intact; provide a launcher/default-config route for platform users. *Why:* Reviewers need to regenerate from the repository without hidden manual copying.

**3. Bundle minimal instructions.** Include controls, options, configuration examples, target system prerequisites and highscore location. Include required asset/dependency license notices and use original or

properly licensed visuals. *Why:* A downloadable game must explain how to launch and operate without opening the repository.

**4. Test outside the source environment.** Copy/extract the artifact to a fresh directory, launch from a different working directory, play, pause, finish, save and relaunch. Test missing optional data, read-only install directory and configured writable storage. *Why:* This distinguishes a self-contained release from a program accidentally using development files.

**5. Publish a free access-controlled build.** Use an appropriate platform such as itch.io. Choose restricted access with reviewer access or public with unlisted search/browse, and verify downloads are available for free. Do not rely on draft mode as delivery. Record URL, access method and artifact version. *Why:* The platform's visibility settings must satisfy private/unlisted access while still allowing review. [S12]

**6. Verify delivery and rebuild.** Download the uploaded artifact as a reviewer would, rerun smoke tests, and rebuild once from the root recipe after reinstalling the assigned wheel. Save checksum/version and observed results in management evidence. *Why:* Testing the local pre-upload folder alone does not prove the published build is correct.

### Practical example

```
# Illustrative commands after the real spec exists:
python3 -m pip install -r requirements.txt
python3 -m PyInstaller packaging.spec
# Extract/copy the resulting one-folder artifact into a clean test directory.
# Launch with its documented config/launcher; save and relaunch.
# Record: source commit, target OS, build command, artifact hash, platform URL.
```

**Expected output:** Root packaging recipe, tested downloadable artifact, in-package instructions and platform/rebuild evidence.

### Definition of done

- ☐ A reviewer can obtain the free unlisted/private build and launch it.
- ☐ The downloaded game includes every required feature and persists scores.
- ☐ Full source and build script/spec are in the repository as required.
- ☐ Both developers can regenerate the tested package and explain its paths.

**Common mistakes:** A draft platform page may not allow downloads. Do not save scores into temporary extraction paths or assume native MLX libraries are automatically collected. Platform access details must be tested with reviewer-like access.

**Integration:** B01 supplies native dependencies, B05 writable storage policy, A02 bundled config, A08 release instructions, and B07 final downloaded-build acceptance.

# Chapter 5 — Project Roadmap

## 5.1 Phases, parallel work and exit gates

Plan for six calendar weeks at roughly 20–24 focused hours per developer each week. The 216 base person-hours are about 108 each; reserve another 20–30% for unknown graphics/packaging issues and beginner learning. The schedule is an estimate, not a deadline from the subject. Move a milestone when evidence changes rather than silently removing mandatory work.

| Phase | Developer A | Developer B | Prerequisite / milestone |
|---|---|---|---|
| 1. Discovery & contracts Week 1, first half | A01: read subject, contracts, environment; begin A08 records. | B01: graphics feasibility; review coordinates, snapshots and errors. | Gate: minimal window, actual wheel call, agreed interfaces and requirement register. |
| 2. Independent foundations Week 1–2 | A02 config and A03 adapter; prove real seeds/walls. | B01 lifecycle, B05 persistence; early B08 minimal package. | Parallel after contracts. Gate: safe settings, valid graph, working storage/window package. |
| 3. First playable slice Week 2–3 | A04 world and A05 movement; begin A06 pickups. | B02 render, B03 input/menu, B04 ghost fixtures. | Gate: real generated level, legal player motion, HUD and one dot scored. |
| 4. Complete rules & loop Week 3–4 | A06 contacts/lives/power; A07 ten-level progression and timing. | B04 chase/flee/wait; B06 cheats and end/name screens. | Pair on contact order. Gate: start → play → win/loss → save → menu, pause freezes all timers. |
| 5. Hardening & evidence Week 5 | Fix owned defects; finish README and decision records. | B07 full acceptance/fault injection; B08 complete packaging recipe. | Gate: R01–R19 covered, exact lint passes, no blocking defects, clean reinstall succeeds. |
| 6. Delivery & rehearsal Week 6 | A08 final evidence, cross-module defense and recode drills. | B08 platform upload/download/rebuild; B07 final artifact smoke. | Gate: reviewer-accessible free build, full repository, both people explain and rebuild. |

## 5.2 Visual timeline

| Workstream | W1 | W2 | W3 | W4 | W5 | W6 |
|---|---|---|---|---|---|---|
| A01 contracts/setup | Build | Review | | | | |
| A02–A03 config/adapter | Start | Build | Verify | | | |
| A04–A05 world/movement | | Start | Build | Verify | | |
| A06–A07 rules/lifecycle | | | Start | Build | Fix | |
| A08 evidence/README | Track | Track | Track | Draft | Finish | Defend |
| B01 graphics + early package | Build | Verify | | | | |
| B02–B03 views/input | | Start | Build | Verify | | |
| B04 ghosts | | Fixture | Build | Integrate | Fix | |
| B05–B06 storage/end/cheats | Store | Store | | End UI | Verify | |
| B07 acceptance | Plan | Fixtures | Checks | Checks | Full | Smoke |
| B08 delivery | Spike | | | | Build | Publish |

Critical path: contracts → validated world → movement/contact rules → lifecycle → complete UI/save loop → acceptance → delivered package. A parallel graphics feasibility path must finish early; if it fails, it can become the critical path. Avoid leaving packaging until the final week: B01’s minimal-window artifact should prove native dependencies in week 1.

## 5.3 Daily execution and risk control

• Start a work session by naming one observable outcome, such as “invalid score JSON opens an empty leaderboard with a warning.” End by recording observed behavior and remaining work.

• Integrate one narrow vertical slice every two or three working days. Do not wait for every menu, animation and ghost style before the first real maze can be played.

• Treat each gate as evidence: attach command output, test result, screenshot or artifact identifier. A statement such as “nearly done” is not a gate.

• When delayed, defer optional graphics polish and additional cheats first. Never defer the ten levels, required screens, external generator, error handling, packaging or README sections.

| Risk | Early signal | Mitigation / owner |
|---|---|---|
| Unsupported graphics binding | Minimal window cannot run/package. | B01/B08 investigate in week 1; audit a permitted substitute before broader UI work. |
| Generator/interface surprise | Constructor mismatch, bad center or wall asymmetry. | A03 tests real wheel immediately; A04 logs documented placement fallback; never patch package. |
| Integration ambiguity | Both modules change score/time or raw key codes leak into model. | A01 contracts; A06/B04 pairing; reject ambiguous PR interfaces early. |
| Time estimation drift | One stream is >20% behind or blocked for more than a session. | A08 updates actuals and redistributes support/review work while preserving ownership. |
| Release-only storage failure | Works in repo, fails in installed directory. | B05/B08 separate assets/config from writable data and test fresh directories. |
| Late evaluation gaps | No management history, unreadable max map, missing menu option. | B07 traceability and A08 records from day one; verify all requirements at each milestone. |

# Chapter 6 — Collaboration and Integration

## 6.1 Branch strategy and module boundaries

Use a small, stable main branch and short task branches such as feat/A03-maze-adapter or feat/B05-highscores. Each branch should contain one reviewable behavior or contract change. Both developers can work concurrently, but each production module has one primary owner. A owns config/maze/world/movement/rules/session; B owns graphics/rendering/input/screens/ghosts/highscores/packaging. Shared contracts and the root launcher need explicit coordination before edits.

Merge a branch when its stated behavior works, relevant tests pass, required lint passes and the other developer has reviewed the contract and at least one failure path. Do not delay a small correct adapter until every feature in a large task is finished. Keep incomplete features off the normal user path rather than merging a broken main branch.

```
git switch main
git pull --ff-only
git switch -c feat/A03-maze-adapter
# Implement and run local QA plus mandatory lint.
git add pacman/maze_adapter.py pacman/contracts.py
git commit -m "Adapt assigned maze wheel and validate reciprocal walls"
git push -u origin feat/A03-maze-adapter
# Open a PR, link R06 and tests T06/T07, and request B's review.
# After merge, both developers update main before starting the next branch.
```

## 6.2 Define contracts before depending on behavior

A03 and B04 agree on neighbors(cell) before coding. B04 can test on a five-cell graph without a window or generator. When A03 is merged, replace the fixture graph with the real adapter in one integration test. Do not change neighbors from cells to directions without updating the contract and both call sites in the same reviewed change.

| Boundary | Producer → consumer | Agreement / failure contract |
|---|---|---|
| Config | A02 → A04/A07/B05 | Only normalized typed values; diagnostics before gameplay; resolved storage path. |
| Topology | A03 → A04/A05/B04 | (x,y), reciprocal legal edges, immutable walls, domain error on invalid output. |
| Tick | A07 → A05/A06/B04 | Seconds, fixed dt, old snapshot, at most one cell step/tick under speed bounds. |
| Ghost result | B04 ↔ A06 | B proposes motion; A resolves contact and score; eaten state set once; own-corner identity retained. |
| Views | A07 → B02/B03/B06 | Read-only snapshots, one authoritative application state, explicit commands and result tokens. |
| Persistence | B06 → B05 → B03 | Validated name/result, save result reporting, no duplicate insertion on retry; top ten shown in menu. |
| Release | B08 ← both | Explicit resources/native dependencies and writable data path; same source behavior in artifact. |

## 6.3 A concrete two-person integration example

**Morning:** A and B agree that power pickup is resolved before contact and ghost proposals include old/new cells. A adds a PowerStarted event and writes a local contact fixture; B implements fleeing from a fixture distance field. Each opens a small PR linked to R09/R10.

**Review:** B spots that A awards a ghost reward again while it waits. A spots that B’s distance cache survives a level change. They add failing regression cases, fix the owner modules, and review the new results. The PR descriptions explain expected behavior, tests and unresolved limits without claiming unrelated features are complete.

**Integration:** Merge the contract first, then each implementation. Run a real seeded map, collect a power pellet, eat a ghost, pause during its wait, resume and verify its own-corner return. Record the seed and the observed pass in acceptance evidence. A then explains the BFS policy back to B; B explains collision ordering back to A.

## 6.4 Review checklist and conflict handling

• Read the requirement and contract before reviewing implementation. Verify behavior, boundary errors, state ownership, complexity and resource cleanup, not just formatting.

• Ask the author to trace one concrete input through the function. For AI-assisted code, both people should be able to justify each state transition and exception branch.

• If a contract change breaks another branch, agree the new signature together and update a small migration note. Rebase only your own unshared work; do not force-rewrite the other developer’s branch history.

• For a merge conflict, inspect both intended behaviors and resolve the logic, then rerun relevant tests. Choosing “ours” or “theirs” blindly is not a resolution.

• Keep disagreement records short: options, evidence, decision, owner and date. Time-box uncertain experiments; choose the simplest option that meets the subject.

## 6.5 Understanding the complete system

Once a week exchange module walkthroughs. A demonstrates graphics allocation, score replacement and packaging paths; B demonstrates normalization, reciprocal walls, contact priority and progression reset rules. Rotate who performs the final acceptance demonstration. Both developers should be able to draw the runtime flow without notes and make a small change outside their primary modules.

# Chapter 7 — Testing and Validation

## 7.1 Test layers and evidence format

Use local unit tests for pure behavior, integration tests for the real wheel/backend and acceptance sessions for the full user journey. The subject says test programs are not submitted or graded; retain the local programs separately and commit the required acceptance plan, bugs and results in project_management. If school submission guidance later differs, resolve that explicitly rather than silently ignoring the subject wording.

Each evidence row should record: test ID, requirement IDs, source revision, environment, input/seed, steps, expected result, observed result, pass/fail and owner. The examples below are a test specification, not results of a game implementation. Only the wheel inspection/probe in §1.3 was executed while making this guide.

## 7.2 Requirements-to-tests traceability

| Test | Req. | Owner | How to test | Expected result |
|---|---|---|---|---|
| T01 | R01,R03 | A01 | Run Python version check, flake8 . and exact required mypy flags; inspect OOP boundaries, type hints and docstrings. | Python ≥3.10; both tools exit 0; owned classes/functions documented and typed. |
| T02 | R02,R03 | A01/B07 | Run install/run/debug/clean; close windows repeatedly; inspect .gitignore; confirm local edge-case test programs exist. | All mandatory targets work; pdb starts main; clean preserves source/config/scores; no leaked resources. |
| T03 | R04 | A02 | Launch with 0 or 2 args, missing file, wrong suffix, invalid encoding and malformed JSON. | Clear message and clean nonzero exit under documented policy; no traceback. |
| T04 | R05 | A02 | Load # comment lines, whitespace-prefixed comments, hash in string, unknown key, missing field. | Only full comment lines removed; unknown keys ignored; missing fields default with warnings. |
| T05 | R05,R11 | A02 | Use bool/float/null for ints, width 0/huge, negative rewards, empty/short levels list, wrong nested object. | Type/range recovery documented; no negative rewards; successful config has ≥10 safe level specs. |
| T06 | R06,R19 | A03 | Reinstall original wheel; call real adapter; fake generation exception and malformed output. | Unmodified assigned package used with perfect=False; valid graph or clear handled failure. |
| T07 | R07 | A03/A04 | Compare two first-level seed-42 runs; log later positive seeds; verify default and rectangular placements/reachability. | First topology repeats; later generated randomly; 4 ghosts/power pellets; central spawn; reachable dots. |
| T08 | R08,R10 | A05/A06 | Walls, turns, equal-time movement, hostile contact/swap, four ghosts at once, central respawn, last-life loss. | Legal cardinal moves; stable speed; exactly one life lost per hostile tick; zero lives ends once. |
| T09 | R09,R10 | B04/A06 | Chase/flee distance, power+contact, ghost eaten once, wait 4.99/5.00 s, own-corner return. | Normal pursuit, edible evasion, exact X/Y/Z scoring, no duplicate reward, timed return. |
| T10 | R10,R11 | A06/A07 | Collect final ordinary/power pellet; compare score/lives before and after next level; finish level 10. | Clear predicate follows documented policy; carryover preserved; final victory once. |
| T11 | R11 | A07 | Pause during movement/power/wait/timer; resume after long wall-time gap; reach exact timeout. | Simulation and all deadlines freeze during pause; resume has no catch-up jump; timeout loss policy holds. |
| T12 | R12 | B05 | Missing/corrupt score files; bad rows; names at lengths 1/10/11; negative/float/bool scores; 11 rows/ties. | Valid top ten sorted; invalid rows handled; zero score allowed; errors do not crash. |
| T13 | R12,R13 | B05/B06 | Win and lose, enter name, submit twice, simulate write failure, relaunch after successful save. | Final score/name route on both endings; one entry per result; honest failure feedback; persistence survives. |
| T14 | R13,R15 | B01/B02/B03 | Walk all menus, inspect HUD at max dimensions, focus loss, close each screen; inspect graphics-call mapping. | All required options/fields visible; controls respond; clean exit; every graphics call has equivalent. |
| T15 | R14 | B06 | Toggle cheats; skip one level with held key; freeze, invincibility and add-life; turn off/new game. | Visible useful controls, one action per press, documented semantics, resets without stale flags. |

| Test | Req. | Owner | How to test | Expected result |
|---|---|---|---|---|
| T16 | R16,R19 | B08 | Build from root recipe, download platform artifact, launch in fresh directory, save/relaunch, regenerate. | Free reviewer-accessible private/unlisted release works; instructions/resources complete; rebuild succeeds. |
| T17 | R17,R18 | A08 | Inspect root README and management directory; verify first line, English sections, AI use, timeline/actuals. | All required sections/evidence present and accurate; links/defaults reflect implementation. |
| T18 | R19 | A08/B07 | Both explain whole flow; reviewer edits config; reinstall generator; perform small code change and focused retest. | Both can explain and adapt; no reliance on patched package or unexplained AI output. |

## 7.3 Concrete configuration and graph cases

| Input / fixture | Expected result | Debugging focus |
|---|---|---|
| {"width": true} inside a level | Warn; use default width 15. | Exact integer check; bool is an int subclass. |
| Level width 0 or 1000000 | Clamp to agreed 7 or 31 with a clear warning; generated topology still validated. | Range bound must match renderer and package tests. |
| JSON string containing # | Retain the string; only a full comment line is stripped. | Do not use split("#") on arbitrary JSON text. |
| Neighbor mask mismatch: east open, west closed | Adapter rejects this output with coordinates in debug details. | Compare reciprocal wall bits; no repair carving. |
| Blocked central anchor fixture | Nearest reachable fallback follows (distance,y,x); log/document it. | Confirm fallback policy with evaluator; do not mutate topology. |
| Ordinary dot in unreachable component | Builder refuses the world or excludes blocked/non-playable cells before placement according to validation policy. | BFS component and pellet-set membership. |
| Same first seed on default map | Equal immutable walls for tested package/runtime; observed baseline 207 reachable cells. | Use positive seed 42; preserve external package version. |

## 7.4 Interaction and timing truth table

| Situation | Expected state change | Important boundary |
|---|---|---|
| Player enters a normal dot | Remove dot; score += X once. | X=0 still consumes the dot. |
| Player enters a power corner shared with a ghost | Remove power; +Y; power becomes active; contact eats ghost; +Z. | Pickup-before-contact ordering must be shared by A/B. |
| Two hostile actors exchange adjacent cells | Lose one life, unless protected/invincible. | Endpoint equality alone misses the event. |
| Four hostile ghosts contact in one tick | Lose exactly one life and apply one respawn/loss. | Stable processing and life_lost guard. |
| Contact a waiting eaten ghost | No collision/reward while EATEN_WAIT. | At ready_at it becomes active at its own corner. |
| Power active, then pause for 30 real seconds | Power remaining simulation duration unchanged. | Refresh real-time baseline on resume. |
| Last collectible and fatal hostile collision in same tick | Loss under this guide’s contact-before-clear terminal priority. | Document the choice; do not emit both outcomes. |
| Timer would reach zero on the current tick | Timeout result before movement for that deadline-crossing tick. | Use the same exact-boundary rule in code and tests. |
| Skip current level with F3 held | One level transition per press; no free pellet score. | Last-level skip goes through the victory/name flow. |

## 7.5 Debugging method for difficult failures

**Maze failures:** record wheel version, dimensions, seed, failing cell and masks; reproduce through the adapter without a window. Verify bounds and reciprocal bits before debugging ghost logic. Never change the external generator to make a test pass.

**Timing failures:** record state, sim_time, level_elapsed, power_until, ready_at and dt. Advance fake time in small exact steps; check just before/at/after deadlines. If pausing changes a timer, find the code path using real time instead of simulation time.

**Interaction failures:** log one tick’s old positions, proposed positions, collected items and emitted events. Reduce to a two-cell swap or one-corner contact. Check that only rules.py changes lives/score and that rendering has no side effects.

**Rendering failures:** compare a known snapshot with its picture, check origin/tile/stride and display dimensions, then inspect resource reuse. A correct graph with an incorrect wall image is a renderer bug, not a pathfinding bug.

**Persistence/release failures:** print a safe resolved storage path and build identifier in debug output. Try a fresh process and working directory, inspect save result, and verify bundled resources/native libraries. Keep detailed developer logs separate from user-facing concise errors.

## 7.6 Evaluation rehearsal

Prepare a short ordered demonstration: launch with the documented command; show invalid config recovery; start the fixed first level; demonstrate movement, dots, power, chase/flee and ghost return; pause and show frozen time; show life loss; use visible cheats to reach later levels and final victory; enter a name; restart and show persistence. Then demonstrate a loss/name route, the required menus, the package download and the root rebuild command.

The subject states that configuration will be updated and the assigned package reinstalled. A package rebuild may be requested. A brief recode task may occur under evaluation guidance; it is not a promised fixed exercise or time limit. No numerical grading breakdown or bonus rubric is supplied, so this guide does not invent one. The explicit failure warning is that an unhandled review crash makes the program non-functional.

## 7.7 Final project validation checklist

• □ Python 3.10+; OOP/modular source; typed functions/variables where applicable; PEP 257 docstrings; required flake8/mypy checks pass.

• □ install/run/debug/clean/lint targets work; .gitignore excludes generated Python artifacts; cleanup does not delete user scores.

• □ Exactly one JSON argument; full-line # comments; clear diagnostics; missing/invalid fields recover; unknown keys ignored; documented defaults and bounds.

• □ Assigned generator used unchanged with perfect=False; reinstall works; first level fixed, later levels random; failures handled.

• □ Reachable central player, four corner ghosts, four corner power pellets, dots in most corridors, legal cardinal movement and central respawn.

• □ Initial default 3 lives; zero lives loses; ghosts chase/flee and return after delay; all three rewards work once and score never decreases.

• □ At least ten levels; timer/timeout policy; score/life carryover; complete-level/game conditions; pause freezes all simulation timers.

• □ All main/pause/win/loss/name views; all HUD fields visible; Instructions and visible useful cheat mode.

• □ Valid top-ten persistent scores; name length/characters and integer score validation; load at start/save at end; corrupt and unwritable files handled.

• □ Every used graphical-library function audited against MLX; native resources cleaned on normal and error exits.

• □ Downloadable free unlisted/private fully working platform build; in-package instructions; root build script/spec; successful clean rebuild.

• □ English README exact attribution and all required sections; dedicated management evidence with actual progress, choices and acceptance outcomes.

• □ Both developers explain all modules, AI usage and trade-offs; review config changes, dependency reinstall and small recode rehearsed.

# Chapter 8 — Understanding and Defending Our Work

## 8.1 Core design questions and answers

### Why did you choose object-oriented design?

State belongs to objects that preserve invariants: a GameSession owns progression and a ScoreStore owns persistence. Composition lets the session use a graph and ghost policies without inheriting a graphics framework. The subject requires OOP and modular reuse; the exact classes are our choice.

### Why separate drawing from gameplay?

Drawing can happen at a different rate than simulation. A read-only snapshot lets us repaint repeatedly without awarding points or moving actors. It also permits rule tests without a native window.

### What is the difference between configuration parsing and validation?

Parsing checks whether text is valid JSON and produces generic Python values. Validation checks types, ranges and meaning; normalization applies documented defaults/clamps. A syntactically valid width=true is still an invalid integer width.

### Why remove whole comment lines rather than everything after #?

The subject requires lines starting with # to be ignored. A # inside a JSON string is ordinary data. Whole-line filtering satisfies the requirement without breaking such strings and preserves line numbers by keeping blank replacements.

### What does wall mask 10 mean?

10 is binary 1010: east bit 2 and west bit 8 are set, so those walls are closed. North and south are open subject to bounds and reciprocal wall validation. The value is wall information, not a tile color or a list of possible moves.

### Why check both sides of an edge?

A maze corridor is undirected. If the east wall of one cell disagrees with the neighbor’s west wall, the player could pass one way but not back, while rendering or BFS assumes symmetry. We reject malformed topology at the adapter boundary.

### What is a perfect maze and why use perfect=False?

A connected perfect maze is a tree with a unique route between each pair of cells. An imperfect maze may have cycles and multiple routes. The subject explicitly requires PERFECT=False; in this wheel the Python parameter is lowercase perfect=False.

### Did you write or modify the maze generator?

No. The adapter calls the assigned wheel unchanged using size=(width,height), reads maze and validates/translates the output. Fixtures are only local tests. We can reinstall the original package because no gameplay relies on patched internals.

### Why not use the package’s shortest_path for ghosts?

That property describes its fixed entry-to-exit route. Ghosts need routes relative to a player whose cell changes. We compute a distance field through the validated graph instead.

### Why BFS rather than DFS, Dijkstra or A*?

Each passage has equal cost, so BFS discovers shortest distances in O(V+E). DFS does not guarantee shortest paths. Dijkstra adds overhead for weighted edges we do not have. A* is useful for one target path; one BFS from the player conveniently serves all four ghosts.

### How does fleeing work and what are its limitations?

An edible ghost chooses a legal neighboring cell with maximum BFS distance from the player. This is a local evasion policy and can oscillate or eventually be caught. It meets the proposed run-away behavior without claiming globally optimal escape. Tie rules are deterministic.

### Why does seed 0 not fix the first level?

The supplied wheel treats a positive seed as deterministic but invokes fresh seeding for non-positive values. We validate the fixed first seed as positive and default to 42. Later levels receive logged random positive seeds.

### Why preserve global random state around generation?

The wheel calls module-level random.seed. Saving and restoring that state prevents generation from unexpectedly changing other random consumers. It is safe for our synchronous single-threaded use, not a substitute for concurrency isolation.

### How do you know every pellet can be eaten?

The world builder places pellets only in the validated reachable component and verifies each one is reachable from the player. It excludes blocked mask-15 cells and keeps ordinary/power sets disjoint. Placement cannot repair or carve generator walls.

### Why use grid movement instead of continuous physics?

The maze is already a graph and the rules need four directions. Grid stepping gives exact legal moves, simple contacts and deterministic tests. Optional pixel interpolation can smooth the picture without changing authoritative cells.

### What happens when the player and a ghost swap positions?

We compare old and new endpoints. Matching destinations or exchanging endpoints counts as contact. Speed bounds limit each actor to at most one edge per tick; faster future movement would require substeps or swept-path detection.

### What happens if a power pellet and ghost are contacted together?

Pickups resolve first, so the power deadline is active before contacts and the ghost can be eaten. This is our explicit ordering decision, tested jointly by the movement/rules and ghost owners.

### How is pause implemented?

The application continues processing events and rendering while GameSession stops advancing simulation time. Power, protection, ghost respawn and level timers all use that same time domain. Resume discards stale real-time accumulator credit.

### Does fixed timestep guarantee perfect real-time behavior?

No. It stabilizes rule increments and reproducibility. Bounded catch-up avoids a frozen UI under long stalls, but dropping backlog makes simulation slower than wall time. We document and test the chosen policy rather than claiming exact wall-clock equivalence.

### How do you prevent duplicate points and duplicate saves?

A pellet is removed before its points are added; an eaten ghost enters a non-colliding wait state before reward. A final result has one token and the UI/store treat submission or retry as one logical record insertion.

### Why does the score never decrease?

Every configured reward is a non-negative integer and only pickup/eaten transitions add it. Death and level change never subtract it. Tests verify both exact event deltas and the monotonic invariant.

### What resets between levels and between games?

Next level resets world, pellets, positions, level timer and temporary effects, preserving score/lives. New game also resets score, initial lives, level index and cheat flags. Respawn after damage uses a separate documented reset policy.

### How are corrupt highscores handled?

Missing or invalid documents produce an empty list and a warning; valid documents with bad rows retain valid rows. We validate name rules and exact integer scores, sort stably descending and keep ten. A failed write returns an honest error while the game remains responsive.

### What does atomic save guarantee?

Writing a temporary sibling file and replacing the destination avoids ordinary readers seeing partial destination contents. It does not automatically provide multi-process locking or total power-loss durability. Those require additional mechanisms beyond this single-process design.

### Why not use an arbitrary graphics/game engine?

The subject permits only graphical-library functions with MLX equivalents. We audit every actual call and expose a narrow backend. Convenience physics, sound or UI functions are not justified merely because the library can create a window.

### How do you handle an unexpected generator or window error?

At the external boundary we translate expected failures into clear domain/user messages and clean resources. Initialization failures exit cleanly; in-session generation errors return to a usable menu. Debug details are retained separately so robustness does not hide bugs.

### How does a packaged build find its files?

Bundled read-only assets, external/configured config and writable persistent scores have different path policies. The root spec/script collects assets and native dependencies; the launcher resolves a default config and writable data location. We test a downloaded copy outside the repository.

### How was work balanced?

A owns configuration, maze/world, movement/rules and lifecycle; B owns graphics/UI, ghosts, storage and release. Ghost pathfinding offsets A’s heavier collision work. There are eight tasks and 108 estimated focused hours per person, with actuals tracked and reviewed.

### Which things are mandatory and which are choices?

Ten levels, four ghosts, external unmodified generator, required UI, persistence, cheats, platform packaging and documentation are mandatory. BFS, 6 s power, 5 s ghost delay, timeout loss, specific keybindings and module names are our documented implementation choices.

### How did you use AI responsibly?

We identify the exact tasks/parts assisted by AI in Resources, independently verify behavior and have the other developer review it. Each owner can trace the code and explain the underlying invariants. Generated output is not evidence until it is tested and understood.

## 8.2 Complexity and trade-offs at a glance

| Operation | Complexity / cost | Reason or alternative |
|---|---|---|
| Maze validation/build | O(V+E) time, O(V+E) storage | Four directions per cell; BFS reachability. E=O(V) for this grid. |
| Distance field | O(V+E), cached until player/topology changes | One field for four ghosts; Dijkstra unnecessary for uniform costs. |
| Ghost decision | O(1) each after distance field | At most four neighbors; four ghosts is a fixed count. |
| Pellet collection | Average O(1) | Set membership/removal; initial placement is O(V). |
| Rendering | O(V) wall/entity iteration plus actual pixel work | Raster cost depends on image dimensions; reuse buffers/static background when useful. |
| Top-ten ranking | O(n log n) for n validated rows | n is small; stable sort is simpler than a heap. Keep only ten output rows. |
| Frame memory | O(screen pixels + V) | Image buffer plus topology/entities; no allocation growth each frame. |

Optimization should follow measured problems. A 15×15 map has only hundreds of cells; clear ownership and correct timing matter more than exotic pathfinding. Cache adjacency and reuse buffers first. Profile a real accepted maximum size before adding complexity. A byte-size cap on config/score inputs is an optional hardening measure; if added, document the limit and recovery policy.

## 8.3 Small-change rehearsal exercises

| Exercise | Likely files | Explain / verify |
|---|---|---|
| Change points per dot through config | config.py, README defaults, rules.py only if schema changes | Validation must reject negatives; one dot adds the new X and HUD reflects it. |
| Show remaining dots in HUD | contracts.py snapshot, renderer.py | Snapshot reads set length; renderer does not count by modifying the world. |
| Change eaten delay from 5 s to 7 s | ghosts.py/settings, instructions if shown | Fake-time boundaries 6.99/7.00; pause still freezes wait. |
| Change ghost tie order | ghosts.py | Only equal-distance choices change; legal edges and chase/flee rule remain true. |
| Add a controlled timeout policy option | config.py, session.py, README | Normalize known choices; define score/life/reset effects; test exact deadline. |
| Tighten name validation | highscores.py, screens.py through shared validator | Load and input use the same rule; max-length and forbidden-character tests pass. |

For each exercise: identify the requirement and invariant, make the smallest change, run the focused test and lint, explain one trade-off, and have the other developer review it. Do not claim you can change any

arbitrary feature in a few minutes; the subject only describes a brief, scoped modification when requested.

## 8.4 Sources and further learning

Local subject requirements take precedence over tutorial suggestions. Subject citations use printed page numbers, which are one less than PDF page position after the cover. Online references below were checked on 27 October 2026; use APIs compatible with your selected Python 3.10+ environment, not features that exist only in a newer documentation version.

### L1 — Project subject: en.subject.pdf, Pacman, version 1.5

Local file, 21 pages. Entire text read. Requirement register §1.4 maps its printed pages.

### L2 — Assigned wheel: mazegenerator-2.1.0-py3-none-any.whl

Local __init__.py, mazegenerator.py and METADATA inspected. Actual constructor and behavior override contradictory examples.

### S1 — Python json: decoding, errors and serialization

https://docs.python.org/3/library/json.html

### S2 — Python random: seed and RNG state

https://docs.python.org/3/library/random.html

### S3 — Python time: monotonic performance clock

https://docs.python.org/3/library/time.html

### S4 — Python typing and docstrings (study links)

https://docs.python.org/3/library/typing.html

### S5 — Python os: replacement and filesystem behavior

https://docs.python.org/3/library/os.html

### S6 — Official 42Paris MiniLibX Linux header: API reference

https://github.com/42Paris/minilibx-linux/blob/master/mlx.h

### S7 — mypy command-line checks

https://mypy.readthedocs.io/en/stable/command_line.html

### S8 — Flake8 invocation

https://flake8.pycqa.org/en/latest/user/invocation.html

### S9 — Red Blob Games: pathfinding study resource

https://www.redblobgames.com/pathfinding/a-star/introduction.html

### S10 — PyInstaller operating modes

https://pyinstaller.org/en/stable/operating-mode.html

### S11 — PyInstaller runtime paths

https://pyinstaller.org/en/stable/runtime-information.html

### S12 — itch.io project access control

https://itch.io/docs/creators/access-control

Additional study keywords: PEP 257, dataclasses, collections.deque, unittest.mock, finite-state machines, fixed-timestep simulation, graph connected components, bit masks, atomic file replacement and Git pull-request review. S4 and S9 are suggested further-reading links; the technical plan is primarily derived from the subject, inspected package and the algorithms explained here.

## 8.5 Guide audit and handoff

This guide covers all nineteen requirement groups with sixteen owned tasks, a dependency plan, test mapping and defense preparation. Developer A: eight tasks / 108 estimated hours. Developer B: eight tasks / 108 estimated hours. Estimates include each owner’s development, debugging, review and testing, and require contingency for unknown native graphics and release work.

Unresolved choices that deserve explicit confirmation during implementation: the graphics binding/target platform; whether configured lives may override 3; center/corner fallback interpretation; whether “all pacgums” includes power pellets; Unicode versus ASCII names; and what to do on malformed top-level config or abandoned sessions. This guide proposes conservative, documented policies rather than inventing subject requirements. There is no source implementation to validate yet.

Input integrity recorded before guide generation:

```
en.subject.pdf SHA-256:
8debad67e6116888cc4614a24399beee9d9b602e76119c134d522e25ea4edbcf
mazegenerator-2.1.0-py3-none-any.whl SHA-256:
98e12f7e989cef03dd4411657d7bee12653804aee87a568eddfaf21436ffd4aa
```

Use this PDF as a working reference. Convert its proposed contracts into reviewed code, its planned tests into observed evidence, and its estimates into an honest actual-progress record. Completion means a playable, reliable, packaged game that both developers can explain, not simply checking off a list of filenames.